Does Zero-Shot Small-Model Post-Correction Help Bengali OCR? A Negative Result Across Eight Models and Two OCR Engines

Arifur Rahman
ID: 20234103222
Department of Computer Science and Engineering
Bangladesh University of Business and Technology
Dhaka, Bangladesh
arifur.rahman.cs04@gmail.com

***Abstract*—Historical Bengali print remains difficult for optical character recognition (OCR), and small language models are a possible low-cost way to correct its output without retraining. We used eight open models of 0.25 to 3.8 billion parameters and a dictionary spell-checker, zero-shot, to correct Tesseract and EasyOCR output on 40 pages printed between 1713 and 1914. No method significantly reduced the character error rate (CER) of either engine. The general-purpose models mostly copied their input, and gradient-based attribution maps showed each copied word relying on its source. The Bengali-pretrained checkpoints we tested regenerated or dropped the text and raised CER above 0.8, also at full numerical precision and with a completion prompt. Given the error-free reference as input, every method still changed it. A common decoding default, a repetition penalty with an n-gram ban, also acts on the prompt; on the same pages it made every copying model appear harmful, raising CER by up to 1.4, even on error-free input. Confidence gating, which keeps only edits to words the OCR engine was unsure of, removed most of the damage, but its one significant gain, 0.004 CER (about 1%) for Qwen3 1.7B on Tesseract output, is negligible and not robust. Fine-tuned and larger models were not tested.**

***Index Terms*—Bengali OCR, post-OCR correction, small language models, zero-shot prompting, historical documents, low-resource NLP, negative results**

I. Introduction

Bengali has about 278 million speakers and ranks seventh by total speakers [1]. Turning its printed records into searchable text remains difficult. The bbOCR evaluation reports an average CER of 0.78 for Tesseract across Bengali document types, compared with 0.59 for the purpose-built bbOCR system [2]. Another study reports Tesseract character accuracy ranging from 37 to 84 percent across document types [3]. Small differences in strokes distinguish many Bengali characters. Conjunct glyphs combine consonants, and vowel signs can sit before, after, above or below the consonant they modify [3]. Worn type, uneven inking and ornate display lettering add to the difficulty of historical books; training corpora for recognition are also small [2].

Post-correction works on the recognised text, so it can be used without modifying or retraining the OCR engine. A text-only correction step could receive output from any engine. Small models are of interest because their lower hardware requirements may suit archives with limited computing resources [4], including national archives, government offices and non-governmental organisations digitising Bengali documents.

Whether language-model correction helps depends on the setting. Task-specific fine-tuning has improved OCR text [5], [6], as have large models given nearly clean English input [7]. Zero-shot use has produced mixed results: open models helped English but not Finnish [8], and only one of four quantised 7 to 13 billion parameter models improved an English newspaper corpus [4]. Failures have been reported with noisier input and less-resourced languages. Even a 70-billion-parameter model raised error rates once input CER reached about 0.15 [7]; zero-shot models also increased the word error rate of Hindi speech-recognition output [9]. Bengali OCR correction has used lexicons, morphology and edit distance [10], [11]. A commercial language model has also cleaned OCR text from modern textbooks during benchmark construction, without a reference-based check of the result [12]. Such cleaning carries a risk: a model can change words that were already right, and an archive that runs one over its scans may make its text worse without knowing it. Evaluations can also mislead in the opposite direction, because a generation setting can make a model that leaves its input alone look destructive. We measure both risks for zero-shot open models below four billion parameters on historical Bengali print, with one correction step applied to the output of two OCR engines. We ask:

- **RQ1.** Can small open language models (0.25 to 3.8 billion parameters), prompted zero-shot, reduce the character and word error rates of OCR output for historical Bengali print?
- **RQ2.** Is the effect of correction consistent across OCR engines when the correction step is not adapted to either engine?
- **RQ3.** What does correction do to the text: does it repair recognition errors, or does it change text that was already

correct?

- **RQ4.** Do the decoding setting, the prompt, the chunking or a safeguard change the outcome?

As prompted, the models did not significantly reduce CER. Their outputs ranged from largely unchanged OCR to regenerated text, and all methods altered some correct text. We report the following contributions:

1) A decoding pitfall for evaluations of correction: a repetition penalty and an n-gram ban, common defaults against repetition loops, also act on the prompt in a decoder-only model, and they make a model that mostly copies its input appear to damage it. We show this on 40 pages for each constraint alone and both together, including on error-free input.
2) A controlled evaluation of zero-shot post-correction on historical Bengali print: eight open models below four billion parameters, two OCR engines and a dictionary baseline on 40 pages whose split was frozen before any model was run, with paired bootstrap intervals, Holm-corrected Wilcoxon tests and equivalence tests.
3) An analysis of what correction changes and whether it can be contained: a clean-text control, correct words broken and wrong words fixed, copying versus regeneration, seen in the outputs and in token-level Grad-CAM maps, precision and prompt-format checks, prompt and chunking variants, page-level safeguards, and edits restricted to words the OCR engine was unsure of.
4) Public code, page lists and prompts.

II. Related Work

The relevant work covers recognition, correction of its output, and Bengali models and tokenisation. We also consider grammar and spelling correction in Bengali and other Indic languages.

A. Bengali OCR systems

Bengali OCR research has mostly addressed recognition itself. bbOCR combines detection, a Bengali recogniser and layout reconstruction to make whole documents searchable; its evaluation compares it only with Tesseract, reporting average text-level CER of 0.59 and 0.78, respectively [2]. Document type matters: Rabby et al. train separate word-segmentation models for computer-composed, letterpress, typewritten and handwritten material, obtaining character accuracy of 83 to 90 percent compared with Tesseract's 37 to 84 percent [3]. Other work operates at different scales. Safir et al. recognise single handwritten words without surrounding context, with CER 0.091 and WER 0.273 [13]. Chaudhury et al. address degraded printed Bangla by separating text from background with a generative adversarial network, then using an encoder-decoder recogniser designed to reduce word error rate [14]. These approaches improve recognition but leave errors on difficult or degraded pages. None supplies a separate correction step for those errors.

B. Post-OCR correction before language models

Nguyen et al.'s survey describes post-OCR methods developed before large language models and notes the poorer recognition quality of historical material [15]. Shared benchmarks came from the ICDAR competitions: about 12 million characters of English and French monographs and periodicals spanning four centuries in 2017 [16], then about 22 million characters in ten European languages in 2019, where the best system improved the text by 44% [17]. Copying is useful in low-resource correction. A model that can copy characters from first-pass OCR reduced error rates by an average of 34% across three endangered languages [18]. Bengali approaches have relied on lexical resources. Pal et al. parse words into roots and suffixes for lexicon lookup [10]; Ahmed et al. use edit distance and character n-grams with about 541,000 dictionary words, lowering error rates by about 3% on 50 documents [11]. Such methods require aligned OCR/reference pairs or language-specific lexicons. We know of no comparable resource for historical Bengali print, hence our interest in models that require no task-specific training.

C. Post-OCR correction with language models

Fine-tuning and zero-shot prompting have produced different outcomes. On short segments of 19th-century English newspapers, a fine-tuned Llama 2 reduced CER by 54.5 percent, compared with 23.3 percent for fine-tuned BART [5]. Without task-specific tuning, success varies by language and input quality. In Kanerva et al.'s English and Finnish evaluation, most open-weight models of 8 to 70 billion parameters improved English, but every one made Finnish worse; only GPT-4o, evaluated alongside them, improved Finnish [8]. Koynov and Doan show how this can change with noise: zero-shot GPT-4o-mini and Llama 3.3 70B lowered newspaper CER by 48 to 58 percent when input CER was 0.07, but raised it by about 19 percent at 0.15. Llama also raised German CER by 41 percent at an input CER of 0.25 [7]. On an English newspaper corpus with input CER 0.084, only one of four quantised 7 to 13 billion parameter models helped; the others increased CER, one five-fold [4]. A study of instruction-tuned German models likewise found no reduction in average periodical CER, although the best model reduced WER by 22 percent. Its reanalysis of an earlier study found the greatest degradation on inputs longer than about 400 characters [19].

The 2026 HIPE-OCRepair task compared zero-shot, fine-tuned and continued-pretraining systems of 8 to 24 billion parameters on English, French and German. The best combined continued pre-training with fine-tuning and outperformed the zero-shot systems; over-correction of accurate text was a recurring failure [20]. Other successful results use different conditions. A frontier model reduced CER on Portuguese survey pages from 11.7 to 4.7 percent [21], while a supervised two-stage system improved historical Bulgarian but was specific to its language and engine [6]. Direct multimodal transcription of 18th-century Russian sometimes beat conventional OCR, yet also introduced archaic characters from the wrong period [22]. Modern-language spelling correction can itself be

unsuitable for historical text: Do et al. attribute part of its poor performance to the mismatch between modern and historical corpora and instead anchor a language model to reference text [23]. In the studies reviewed here, zero-shot correction helped mainly with large or frontier models and input CER below about 0.15. Damage to already accurate text remains a common problem.

D. Bengali language models and tokenisation

Bengali models differ in whether they can generate text at all. BanglaBERT is an encoder for understanding tasks and cannot rewrite OCR output [24]; BanglaT5 is a sequence-to-sequence generation model [25]. The TituLLMs family uses Bengali-pretrained Llama 3.2 models at 1B and 3B, trained on about 37 billion tokens with an extended tokenizer [26]. BanglaLlama continues pretraining other Llama models. Its authors report that Bengali and Hindi together account for less than 0.21 percent of Llama 2's pretraining tokens, compared with 89.7 percent for English [27]. Limited exposure is only part of the concern. Mahfuz et al. find that inefficient Bengali tokenisation increases computation and reduces generation quality when assessing the need for dedicated Bengali models [28]. Across languages, Petrov et al. show that English-centric tokenizers require several times more tokens for equivalent content; Bengali needs roughly five to ten times the English count, compared with about 1.4 to 1.6 times for multilingual XLM-R and M2M100 tokenizers [29]. Ahia et al. also associate over-segmentation with higher cost and poorer model performance [30]. General multilingual models therefore face both limited Bengali exposure and inefficient tokenisation. To our knowledge, Bengali-pretrained models have not been evaluated on OCR correction.

E. Correction of Bengali and other Indic text

The closest Bengali language-model correction study concerns grammar rather than OCR. Instruction-tuning improved its scores by 3 to 7 points over zero-shot use, though humans still did better. Mixing the older literary register with the modern register was among GPT-4o's hardest error categories [31]. In Hindi speech-recognition correction, fine-tuned models of 0.3 to 0.58 billion parameters outperformed much larger models. Zero-shot GPT-4o-mini increased WER from 28.6 to 31.8 and from 18.0 to 25.1 percent; the authors attribute this to over-correction that grows with model size [9]. Supervised Bengali spelling systems include BSpell, which combines BERT and a convolutional sub-network and outperforms earlier checkers on two Bengali datasets and one Hindi dataset [32], and a transformer detector-purificator-corrector for Bengali and other resource-scarce Indic languages [33]. BnMMLU provides an example of language-model use after Bengali OCR: GPT-3.5 copy-edited EasyOCR output from modern textbooks and exam guides during dataset preparation. The authors report fewer formatting and spelling errors and manually check about 10% of the items, but give no reference-based error rate for that step [12]. These studies cover modern grammar and spelling, or unmeasured cleaning of modern print. We found no evaluation of language-model OCR correction on historical Bengali print.

F. Research gap

Table I compares the language-model studies. Existing Bengali OCR correction uses lexicons, morphology and edit distance [10], [11], or a commercial model for cleaning without reference-based evaluation [12]. None of the reviewed work measures zero-shot language-model correction of historical Bengali print. The reviewed post-OCR studies also do not evaluate open models below seven billion parameters, input CER above 0.25, or one correction step across OCR engines. Our evaluation addresses these gaps in language, scale and engine coverage.

III. Methodology

We score raw and corrected OCR against the same ground truth. The protocol below specifies the corpus, recognition and correction steps, model selection, classical baseline and evaluation.

A. Overview

Each page passes through two OCR engines, then through a correction model in chunks (Fig. 1). We score both versions of the text against its manual transcription. Keeping recognition and correction separate lets the same correction step accept output from any engine. All models are used as released, without training or fine-tuning.

B. Dataset

We use the ground-truth transcriptions released with the ICDAR 2019 Competition on Recognition of Early Indian Printed Documents (REID2019) [34], produced by the British Library's Two Centuries of Indian Print project with the PRImA Research Lab and Jadavpur University. We use version 2 of the British Library's release, whose XML coordinates were improved [35]. The source volumes are Bengali books printed between 1713 and 1914. The digitised images are out of copyright and the transcriptions are released under the Public Domain Mark.

The release contains 81 page images with PAGE-XML annotations. Twenty-nine of them (36%) have text regions marked but no transcribed text, so no error rate can be computed, and they are excluded; 52 pages remain. We drop the two pages with the thinnest transcriptions, measured by the share of text regions that actually contain text, which leaves a pool of 50 pages. The pool is split into 10 development pages and 40 evaluation pages, and the split was frozen in saved page-identifier lists and committed to version control before any correction model was run. The development pages were used only to debug the pipeline and as the source of the worked examples in one prompt variant; no development page contributes to any reported score. All 40 evaluation pages are scored (Fig. 2). They come from 37 books, with at most two pages from any one book (Table II). The release gives no publication date per book. Each page was labelled by its main

TABLE I
LANGUAGE-MODEL CORRECTION OF RECOGNITION OUTPUT: PRIOR WORK AND THIS STUDY

Study	Language	Task	Models (size)	Setting	Input error	Did correction help?
Thomas et al. [5]	English	post-OCR	Llama 2 (7B, 13B); BART	fine-tuned	CER 0.08	Yes: CER −54.5% (13B)
Danilova & Aangenendt [19]	German	post-OCR	Llama 2 13B, Mistral 7B; BART	fine-tuned	CER 0.02	No CER gain on average; WER −22% (best)
Kanerva et al. [8]	English, Finnish	post-OCR	open models 8B–70B; GPT-4o	zero-shot	CER 0.07 / 0.09	English: 6 of 7 models; Finnish: only GPT-4o
Koynov & Doan [7]	English, German	post-OCR	GPT-4o-mini; Llama 3.3 70B	zero-shot (+ fine-tuned)	CER 0.07–0.25	Yes at CER 0.07; worse at CER $\geq$0.15
Vershinin et al. [4]	English	post-OCR	7B–13B, 4-bit quantized	zero-shot	CER 0.08	1 of 4 models improved CER
Beshirov et al. [6]	Bulgarian	post-OCR	BERT detector + LSTM corrector	supervised	—	Yes
Ehrmann et al. [20]	English, French, German	post-OCR	8B–24B	zero-shot; fine-tuned	CER 0.004–0.057	Fine-tuned: yes; zero-shot: mixed (worse than raw OCR on French)
Viana [21]	Portuguese	post-OCR	GPT-5.5 (frontier)	zero-shot	CER 0.12	Yes: CER −60%
Bhattacharyya & Bhattacharya [31]	Bengali	grammar correction	1.1B–8B; GPT-4o	zero-shot; fine-tuned	clean modern text	Fine-tuned 3–7 points above zero-shot
Kumar et al. [9]	Hindi	post-ASR	Llama 1B–70B, GPT-4o-mini; mT5, ByT5 (0.3–0.58B)	zero-shot; fine-tuned	WER 0.18–0.29	Zero-shot: worse; fine-tuned small models: yes
This study	**Bengali**	**post-OCR**	**0.25B–3.8B; SymSpell**	**zero-shot**	**CER 0.30–0.34**	**No significant CER reduction; at most −0.004 with confidence gating**

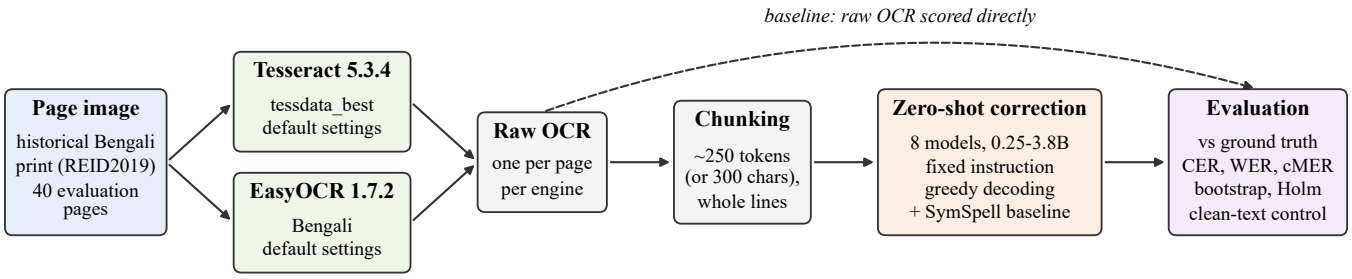


Fig. 1. Pipeline of the study. Each page image is read by two OCR engines. The raw text is chunked and corrected zero-shot by one of eight models, or by a dictionary spell-checker, and both raw and corrected text are scored against the ground truth. The dashed arrow marks the uncorrected baseline: raw OCR scored directly.

content, from its transcription, by a single annotator without a second check: 10 title pages, often in ornate display type, 8 verse pages (including verse followed by a commentary), 13 prose pages, and 9 lists (tables of contents, indexes, tables and dictionary entries); the labels are released with the code. A random sample of 15 of the evaluation pages (seed 403) was also processed on a CPU; it provides the CPU timings and an earlier decoding comparison.

TABLE II
EVALUATION PAGES

Pages	40
Books	37 (at most 2 pages per book)
Printed	1713–1914 (no per-book dates)
Page types	10 title, 8 verse, 13 prose, 9 list
Reference characters per page	median 577 (range 193–2715)
Reference words per page	median 89
Raw Tesseract length / reference	mean 1.03, median 1.00
Raw EasyOCR length / reference	mean 1.02, median 0.99

Book: page identifier without its page number. Lengths after Unicode NFC normalisation and whitespace collapsing.

C. Baseline OCR

Two open-source engines produce the noisy text. Tesseract 5.3.4 [36], [37] runs with its LSTM recogniser and the Bengali `tessdata_best` model, and EasyOCR 1.7.2 [38] with Bengali enabled. Both use their default settings, which for Tesseract means automatic page segmentation (mode 3), and receive the page image as released (colour scans at 400 dpi, about 1,800 by 3,000 pixels), without preprocessing. Both ran on a free Google Colab CPU runtime in October 2026. Each engine produces one hypothesis per page. Two engines with

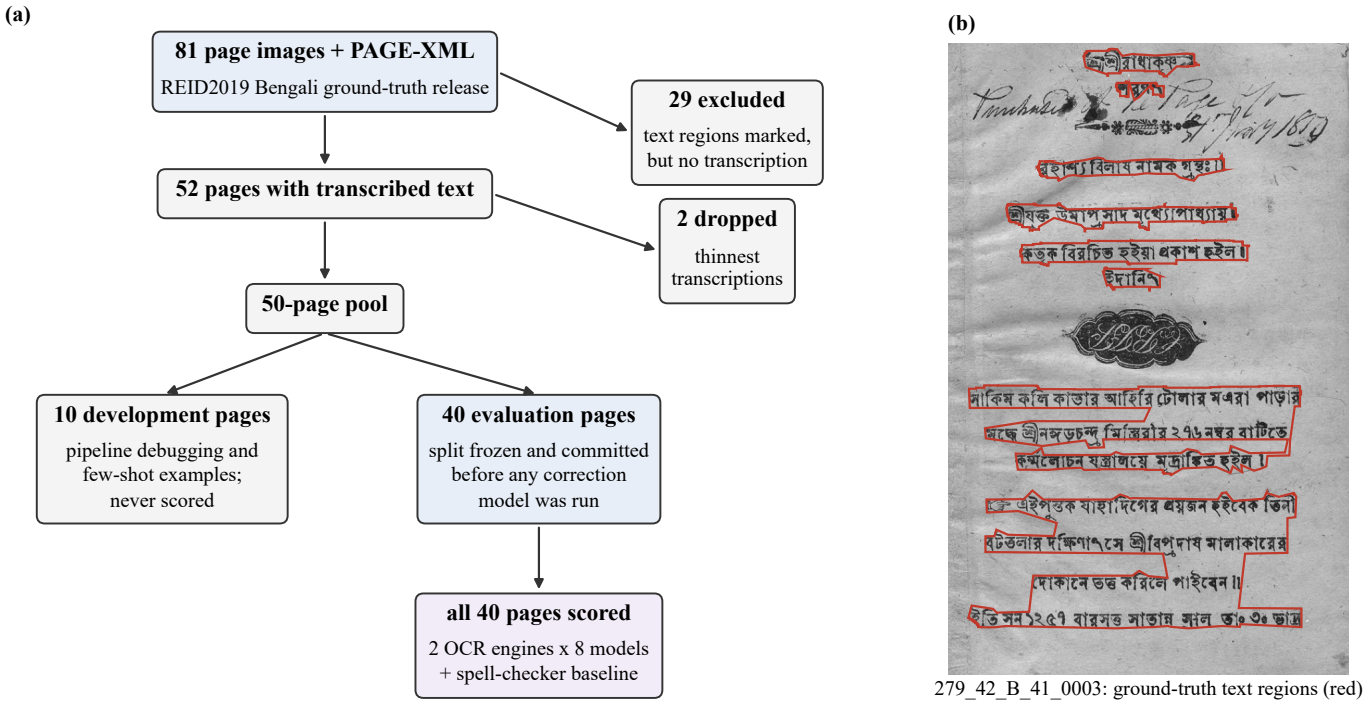


Fig. 2. Dataset. (a) How the 81 pages of the release become the 40 evaluation pages. (b) A sample evaluation page, with the ground-truth text regions outlined in red.

different error profiles let us test whether correction behaves differently depending on which engine produced its input.

The engines read the whole page, while the ground truth covers only the annotated text regions. Text outside those regions, such as stamps, shelf marks or handwriting (Fig. 2b), therefore counts as inserted characters. The reference joins the region transcriptions in the order in which they appear in the PAGE-XML file; 30 of the 40 files also declare a reading order, and on 17 pages it differs from the file order. The OCR text keeps each engine's own reading order and is not matched to the reference, so a different order also counts as errors. The raw OCR is on average 1.03 (Tesseract) and 1.02 (EasyOCR) times the length of the reference (Table II), but this does not show that out-of-region text is negligible, because missing in-region text can offset it. Nor need the mismatch cancel in the paired differences: a model may delete, keep or rewrite out-of-region text differently from the rest. We therefore repeat the main comparison against a reference in the declared reading order, and with a word-recall measure that ignores word order and extra text (Section IV-H). We did not run a region-level evaluation. OCR output is not stable across installations: an earlier run of Tesseract on the 15-page sample, with an unrecorded Colab build from August 2026, gave different text on all 15 pages (mean CER 0.364 against 0.320), while EasyOCR changed little. All reported pages therefore use one OCR run; only the CPU timings and the earlier decoding comparison on the 15-page sample use the August text.

D. Correction layer

Zero-shot. Each model is used as released, without training on correction examples, and the main prompt contains no worked examples. The decoder models receive an instruction and the OCR text; BanglaT5 receives the text alone, as detailed below. We chose this setting because we know of no large public set of Bengali OCR error pairs for training, and because it avoids the cost of task-specific training.

Prompt. The decoder models receive the same instruction, through each model's chat template:

> *The following text was produced by OCR on a historical Bengali document and may contain recognition errors. Correct the OCR errors and output only the corrected Bengali text, with no explanation, preamble, or additional commentary.*

The OCR text follows the instruction. BanglaT5 is not instruction-tuned and has no chat template, so it receives the OCR text alone and is run as a text-to-text model. This prompt was fixed in advance and never revised in response to model output.

Prompt variants. For the four general-purpose models we also test three variants, each changing one thing: the same instruction followed by "Preserve the historical spelling of the original text; if you are not sure whether something is an error, leave it unchanged"; the instruction translated into Bengali; and the instruction with three worked examples. The examples are single lines from three different development pages, chosen by a fixed rule: the first reference line of 30 to 90 characters

whose best-matching OCR line has a CER between 0.05 and 0.30 and contains no Latin letters, taking the engines in turn.

Chunking. Following Kanerva et al. [8], each page is divided into chunks of about 250 tokens of the model's own tokenizer before correction. Chunks are formed by grouping complete lines, so no chunk boundary falls inside a line. Because the tokenizers differ, the resulting chunks range from 168 characters on average for Llama 3.2 1B to 483 for BanglaT5 (Table III). As a model-independent alternative for the four general-purpose models, we also group lines into chunks of at most 300 characters, which every model then receives identically; a single longer line would form its own chunk, but none occurred on our pages. A page whose raw OCR is shorter than ten characters would be left unchanged, since the model would have almost nothing to work from; no evaluation page triggered this rule.

Decoding. Decoding is greedy, so output is deterministic. The maximum number of new tokens is $\min(512, \max(64, 1.5n))$, where n is the token length of the input chunk, and an output that reaches this limit is counted as truncated. No other constraint is applied in the main condition. A repetition penalty and a ban on repeated n-grams are common defaults against degenerate loops, and we evaluate them, alone and together, as a decoding ablation (penalty 1.3; repeated four-grams banned; both together are *guarded* decoding; Section IV-D). In the Transformers library [39] they are implemented by `RepetitionPenaltyLogitsProcessor` and `NoRepeatNGramLogitsProcessor`, which `generate` applies to the whole token sequence; for a decoder-only model this includes the prompt, and therefore the OCR text being corrected, as the library documents [40]. They penalise every token the model copies from its input and forbid it from reproducing any four-token run of that input. Since correction consists mostly of copying, plain greedy decoding is the main condition. In the library source we inspected (version 5.14.1; the GPU runs used 5.19.0), `generate` constructs the penalty processor without a prompt length, so the prompt is included; it can be excluded only by constructing the processor by hand.

E. Choice of models

We evaluate eight models of 0.25 to 3.84 billion parameters (Table III). Parameter counts are computed from each checkpoint's configuration, with tied input and output embeddings counted once.

- *General-purpose models.* Gemma-2B-it [41] (Gemma 1, 2.51B parameters), Qwen3 1.7B [42], Phi-4-mini [43] and Llama 3.2 1B [44] are open instruction-tuned multilingual models; Qwen3 and Phi-4-mini were released in 2025. Bengali is a very small share of the pretraining data of general models [27]. Qwen3 can produce a reasoning trace before its answer; we disable this through its chat template, so it answers directly.
- *Bengali-pretrained checkpoints.* TituLLMs 1B [26] (checkpoint v1.1) is Llama 3.2 1B continually pretrained on Bengali text. It is a base model without instruction tuning and keeps the Llama vocabulary; the template shipped with its tokenizer only concatenates the messages, so it received our instruction and the OCR text as plain text to continue, with no cue for where a corrected version should begin. As a check, we also gave it a completion prompt without template, which states that the corrected text follows the OCR text and ends with "Corrected text:" (Section IV-J). We also evaluate the instruction-tuned TituLLMs checkpoints at 1B and 3B parameters (v2.0 Instruct), which use an extended vocabulary of 170,497 tokens; their model cards give no prompt format, and their shipped template wraps the prompt in an Alpaca-style instruction and response frame, which we used. BanglaT5 [25] (about 248M parameters) is a sequence-to-sequence model pretrained only to restore corrupted text spans and released without task fine-tuning. Without a task prefix it acts as a denoiser rather than a corrector, so we report it as a control. Its model card also asks for inputs passed through the authors' text normaliser, which the main runs did not apply; a rerun with it is reported in Section IV-J.

These results therefore describe these checkpoints with these prompts, not Bengali pretraining in general. BanglaBERT [24] was not considered because it is an encoder and cannot generate corrected text. All models are loaded with the Transformers library [39].

TABLE III
MODELS AND CONFIGURATION

Model	Checkpoint@revision	Params (B)	GPU prec.	Input	Chars/ chunk
Gemma-2B-it	`google/gemma-2b-it@9698841`	2.51	fp32	template	312
Qwen3 1.7B	`Qwen/Qwen3-1.7B@70d244c`	1.72	fp32	template	190
Phi-4-mini	`microsoft/Phi-4-mini-instruct@cfbefac`	3.84	fp16	template	389
Llama 3.2 1B	`meta-llama/Llama-3.2-1B-Instruct@9213176`	1.24	fp32	template	168
BanglaT5	`csebuetnlp/banglat5@76b135c`	0.25	fp32	text	483
TituLLMs 1B	`hishab/titulm-llama-3.2-1b-v1.1@521da71`	1.24	fp32	template	168
TituLLMs 1B Instruct	`hishab/titulm-llama-3.2-1b-v2.0-Instruct-v1.0@039dbd2`	1.32	fp32	template	422
TituLLMs 3B Instruct	`hishab/titulm-llama-3.2-3b-v2.0-Instruct-v1.0@6a96004`	3.34	fp16	template	422

Checkpoint: Hugging Face model and revision. Parameters from each checkpoint's configuration, tied embeddings counted once. GPU prec.: precision on the T4 (fp16 for the two models whose fp32 weights exceed its memory). Input: the prompt through the chat template shipped with the checkpoint, or the OCR text alone. Chars/chunk: mean characters per chunk of about 250 tokens on raw Tesseract text. Decoding for all models: greedy, at most $\min(512, \max(64, 1.5n))$ new tokens, no repetition penalty or n-gram ban.

F. Classical baseline

To test whether the outcome is specific to language models, we also correct the OCR with a dictionary spell-checker.

TABLE IV
TOKENISATION AND COMPUTATIONAL COST

Model	Tokens/ char	Tokens/ word	GPU weights (GB)	CPU min/page	GPU min/page
Gemma-2B-it	0.60	3.9	10.0	10.4	0.38
Qwen3 1.7B	1.08	7.1	6.9	14.3	0.58
Phi-4-mini	0.45	3.0	7.7	—	0.31
Llama 3.2 1B	1.22	8.0	5.0	9.5	0.37
BanglaT5	0.29	1.9	1.0	2.2	0.05
TituLLMs 1B	1.22	8.0	5.0	12.5	0.59
TituLLMs 1B Instruct	0.38	2.5	5.3	—	2.44
TituLLMs 3B Instruct	0.38	2.5	6.7	—	0.32

Tokens/char and tokens/word: each tokenizer on the 40 reference texts. GPU weights: parameters $\times$ 4 bytes in fp32 or 2 in fp16; peak memory during generation was not recorded. GPU: NVIDIA T4, plain decoding, the 40 pages of the main setting. CPU: free Colab CPU runtime in bfloat16, plain decoding, on the 15-page sample with its August 2026 OCR, a different input from the main setting (—: not run on CPU). Times are per page and engine.

SymSpell [45] replaces every Bengali word that is not in its lexicon with the lexicon word at the smallest edit distance, breaking ties by frequency, if one exists within an edit distance of 2; known words, punctuation, digits and Latin text are left unchanged. The lexicon is the 50,000 most frequent words of the Bengali OpenSubtitles corpus [46], of which 44,335 entries consist only of Bengali letters and signs. Its settings are the library defaults, fixed in advance.

G. Computing setting

The motivating setting is CPU-only use by institutions that process Bengali documents without GPU infrastructure. All the models are small enough to run on a CPU. Table IV reports the CPU time per page for the five that were run on a free Google Colab CPU runtime in bfloat16, with plain decoding, on the 15-page sample and its August OCR text, so these times describe a different input from the main runs. To evaluate all 40 pages, eight models and the variants within a free compute budget, the reported runs use a free NVIDIA T4 GPU on Kaggle, in float32, or in float16 for the two models above 3B parameters, which do not fit the GPU's 15 GB in float32. A float32 rerun on two GPUs, of TituLLMs 3B Instruct on all inputs and of Phi-4-mini on Tesseract text, gave practically the same CER (Section IV-J). On the pages where a CPU and a GPU run received identical input, the outputs of the general-purpose models were identical or nearly so, and the scores of the other models agreed (Section IV-I). No paid service is used at any stage.

H. Metrics

Before scoring, reference and hypothesis are normalised to Unicode NFC and runs of whitespace are collapsed, because Bengali conjuncts and vowel signs can be encoded as different code-point sequences that look identical. Zero-width joiners and non-joiners are kept, and the danda counts as an ordinary character. The character error rate is CER $= (S + D + I)/N$, where S, D and I are the numbers of substituted, deleted and inserted characters in the alignment of the hypothesis with the reference and N is the number of reference characters; the word error rate (WER) is the same quantity over words. Both are computed with jiwer [47] for each page and averaged over pages, so every page counts equally regardless of its length; the pooled CER divides the total number of character errors by the total reference length. CER counts code points, so an error in one sign of a conjunct counts once although the reader sees the whole cluster as wrong; the grapheme-cluster CER is computed over extended grapheme clusters, which keep a conjunct and its vowel signs together. CER and WER are unbounded, since an output with many insertions can score above 1. We therefore also report the character and word match error rates, cMER $= (S + D + I)/(H + S + D + I)$ and its word-level counterpart wMER, where H is the number of matched units; both are bounded in $[0, 1]$ [20]. Three further normalisations test the scoring choices: mapping ASCII digits to Bengali digits, removing zero-width joiners and non-joiners, and writing khanda-ta as the single code point U+09CE.

I. Statistics

For each engine and correction method, the unit of analysis is the per-page difference between the corrected and the uncorrected error rate. We report the mean difference with a 95% percentile interval from a paired bootstrap of the 40 pages (10,000 resamples, seed 403), and the number of pages whose error rate fell or rose. We also run a two-sided Wilcoxon signed-rank test [48] on the same differences. Pages whose difference is exactly zero, which are common when a method leaves the OCR unchanged, are dropped before ranking, the default of the test, so we report how many pages remain. The primary family is the 36 CER and WER tests (nine methods, two engines, two metrics), adjusted with the Holm procedure [49], which follows the reporting of Viana [21]. Every other test in this paper is exploratory: each table that reports one names its own Holm family, and Holm controls the error only within a family. Because pages from the same book are not independent, we repeat the bootstrap resampling whole books instead of pages. For the four general-purpose models, whose claim is that they leave the error rate unchanged, we also test equivalence with two one-sided paired t-tests (TOST) against margins of ± 0.01 and ± 0.02 CER.

The evaluation split was frozen before any model ran, and development pages are never scored. We used this one fixed evaluation set and did not perform cross-validation. No model was trained, and greedy decoding is deterministic, so runs were not repeated with different seeds.

J. Analyses

Sensitivity to the reference. Two checks address the mismatch between whole-page OCR and region-level ground truth. First, we rebuild each reference with its regions in the reading order declared in the PAGE-XML file, where there is one (regions it does not list follow in file order), and repeat the paired comparison of CER. Second, we compute reference-word recall: the share of reference word tokens that also occur in the output, counted as a multiset. It ignores word order and does not penalise extra words, so neither a different reading

order nor OCR text from outside the annotated regions lowers it; it does not reward repairs that leave a word wrong.

Clean-text control. To separate over-correction from the difficulty of noisy input, each model also receives the reference text itself in place of the OCR, with the same prompt and decoding. Any change it makes can then only be damage.

Word-level and edit analysis. We align the ground-truth words with the raw OCR and with the corrected output. A ground-truth word that the raw OCR matched exactly and the output does not is a *broken* word; a word the raw OCR missed and the output matches is a *fixed* word [9]. The *change ratio* is the CER of the output measured against the raw OCR, divided by the CER of the raw OCR against the ground truth [7]; a ratio above 1 means the model changed more than there was to repair. *Invented runs* are runs of at least six characters that the output contains and the raw OCR does not [7]. We also report output length relative to the reference, the share of Bengali, Latin and other characters, ground-truth word retention, edit operations against the reference, the error rate by character class and by page type, and the truncation rate.

Attribution. To see which part of its input each output token relied on, we adapt Grad-CAM [50] to token positions. Grad-CAM weights the feature maps of a CNN layer by the gradient of a target score; here the score is the logit of each token the model generated (plain greedy decoding, the main prompt), and the map runs over the token positions of a layer's hidden states. We compute the literal version, which averages the gradient over positions, and an element-wise version, which multiplies gradient and hidden state at each position, as HiResCAM does [51], at mid depth and at three-quarter depth. The first token, special tokens and the predicting position are left out, because they take a large share of the map without saying anything about the context. For each generated token we record the share of the map on the OCR text, on the rest of the prompt and on the model's own earlier output; for a token that copies an OCR token we record whether the map peaks within one position of that source token. We scored 24 generated tokens in one chunk from each of 10 pages (seed 403), on Tesseract text and on clean text, for six models; Phi-4-mini and TituLLMs 3B Instruct did not fit the time of the GPU session.

Output filter not applied. Kanerva et al. strip leading and trailing text that does not align with the input, such as explanatory preambles [8]. We did not apply this step, so all scores use unfiltered model output and any preamble or commentary counts as error; Section VI discusses what this means.

K. Safeguards and confidence gating

Page-level safeguards. As post-hoc checks, we replace a model's output by the raw OCR whenever its length falls outside 0.35 to 2.8 times the input length; these thresholds are taken from a zero-shot system in the HIPE-OCRepair task [20] and are not tuned on our pages. A second rule, an edit-distance cap, keeps the output only if its CER measured against the raw OCR is at most τ, for $\tau \in \{0.05, 0.1, 0.2, 0.3\}$, a grid fixed in advance. Each safeguarded result is compared with the raw OCR by the same paired bootstrap and Wilcoxon test, with Holm correction over the 18 length-rule tests and, separately, the 72 edit-cap tests.

Confidence gating. Both engines report how confident they are: Tesseract per word (0 to 100) and EasyOCR per detected text box (0 to 1), which every word in the box inherits. We re-ran both engines with the same versions to obtain these values; the re-run reproduced the stored OCR text on all 40 Tesseract pages and 37 EasyOCR pages, and confidences are mapped onto the stored words by word alignment. On the three EasyOCR pages that were not reproduced, as elsewhere, a stored word without a counterpart in the re-run counts as confident, so the gate never lets a model change it. The existing model outputs are then gated: after aligning the OCR words with the output words, an edit that replaces or deletes OCR words is kept only if every OCR word it touches has a confidence below a threshold, and an inserted word only if a neighbouring OCR word does; all other words stay as the OCR read them. The model therefore still corrects whole chunks with their context, but may change only what the engine was unsure of. The thresholds are fixed in advance: 50, 70 and 90 for Tesseract and 0.5, 0.7 and 0.9 for EasyOCR.

L. Reproducibility

The page lists, page-type labels, prompts, few-shot selection rule, and the OCR, correction, baseline, scoring and statistics code are public [52], with the frozen split committed before any model was run. Table III names each checkpoint and the revision used; each model's own chat template was used as shipped with that revision, with Qwen3's reasoning trace disabled, and the generation settings are those given under *Decoding* in Section III-D. The GPU runs (10–12 October 2026) used Transformers 5.19.0, the OCR used EasyOCR 1.7.2 and Tesseract 5.3.4, and scoring used jiwer 4.0.0 under Python 3.11. The PyTorch and CUDA versions of the Kaggle and Colab images used for the main runs were not recorded; the precision, prompt-format, decoding and attribution runs (16 October 2026, two T4 GPUs) used PyTorch 2.11.0 with CUDA 12.8 and Transformers 5.19.0. An environment record in the repository lists the known versions, the local analysis environment, and the summary statistics behind every table. Nothing is trained and no paid service is used, so the study can be repeated end to end on free computing resources.

IV. Results

Each comparison pairs a corrected page with its raw OCR. We report the main results and clean-text control, followed by decoding, prompt and chunking checks, comparisons across engines and pages, and statistical robustness.

A. Baseline OCR performance

EasyOCR has lower error rates than Tesseract on all three baseline metrics (Table V); CER is 0.297 rather than 0.341. Tesseract's high error rate is consistent with earlier Bengali

TABLE V
UNCORRECTED OCR BASELINES (40 EVALUATION PAGES)

Engine	CER	WER	cMER	wMER
Tesseract	0.341	0.827	0.290	0.652
EasyOCR	0.297	0.757	0.260	0.629

cMER, wMER: character and word match error rates, $(S+D+I)/(H+S+D+I)$, bounded in $[0, 1]$.

evaluations [2], [3], although we found no published comparison of these engines on historical Bengali print. Neither gives clean input for correction: even EasyOCR has WER 0.757.

B. Effect of correction

The main, ungated methods do not significantly reduce CER for either engine (Table VI and Fig. 3). Confidence gating gives one small exception, discussed in Section V-E. After Holm correction, 23 of the 36 CER/WER comparisons are significant; 22 indicate increased error.

The methods fall into two groups. Gemma-2B-it, Qwen3 1.7B and Phi-4-mini stay close to the raw OCR: their CER changes lie between -0.004 and $+0.017$, and none is significant. Not significant is not the same as unchanged, however (Table VII). Equivalence within ± 0.02 CER is supported only for Qwen3 1.7B on Tesseract output and Gemma-2B-it on EasyOCR output, and within ± 0.01 for none. The other changes are neither significant nor shown to be negligible, and Phi-4-mini raises the bounded wMER significantly on both engines. Each of these models makes some pages better and others worse: Gemma-2B-it 16 better and 16 worse of the 40 Tesseract pages, Qwen3 1.7B 22 and 13. Llama 3.2 1B raises CER slightly but significantly, from 0.341 to 0.380 on Tesseract output and from 0.297 to 0.330 on EasyOCR output ($p = 0.002$ and $p = 0.017$), and worsens 29 of the 40 pages on each engine. The dictionary baseline raises CER by 0.037 and 0.040 and WER by 0.091 and 0.095 (all $p < 0.001$), on 39 and 38 pages. The four Bengali-pretrained checkpoints raise CER by between 0.52 and 1.34 on 38 to 40 pages: BanglaT5 to 0.860 and 0.831, TituLLMs 1B to 1.084 and 1.033, TituLLMs 3B Instruct to 0.958 and 0.886, and TituLLMs 1B Instruct to 1.642 and 1.633. The instruction-tuned 1B checkpoint is therefore worse than the base checkpoint, although the two received differently formatted prompts (Section III-D).

The one significant improvement is Qwen3 1.7B on Tesseract output, whose WER falls from 0.827 to 0.799 (-0.028, 95% CI $[-0.047, -0.013]$, $p = 0.002$). Its grapheme-cluster CER also falls, from 0.431 to 0.417 (-0.015, $[-0.032, -0.004]$, $p = 0.021$ after Holm correction over 18 tests), but its code-point CER does not change significantly (-0.004). Qwen3 thus repairs some whole words on Tesseract text without lowering the overall character error rate; on EasyOCR text it shows no improvement on any metric.

C. Clean-text control

Giving the models reference text removes any recognition error they could repair. Nevertheless, every method changes it (Table VIII). Gemma-2B-it and Qwen3 1.7B do the least damage, with CER 0.032 and 0.035, leaving 6 and 2 of the 40 pages exactly unchanged. The corresponding CERs are 0.060 for Phi-4-mini, 0.057 for the dictionary baseline and 0.119 for Llama 3.2 1B. The dictionary replaces about 31 correct words per page, giving WER 0.268. The Bengali-pretrained checkpoints reach CERs of 0.83 to 1.57 despite the error-free input. These changes are over-correction by construction; their magnitude gives nearly the same ordering of methods as the main evaluation. Phi-4-mini's pooled CER (0.097) is far above its mean (0.060) because of one long page: partway through a chunk its output switches to English prose about physical properties of an object and runs to the token limit, at 1.94 times the reference length, which gives 70% of its character errors on the 40 pages.

D. Effect of the decoding setting

Table IX and Fig. 4 compare plain greedy decoding with the repetition penalty alone, the four-gram ban alone and both together (guarded decoding), for the four general-purpose models on the 40 pages, with the OCR text of both engines and with the clean text as input. Every constraint, alone or combined, raised CER for every model on every input; all 33 comparisons with plain decoding are significant after Holm correction. Guarded decoding raised Gemma-2B-it's CER from 0.353 to 0.545 on Tesseract text, Qwen3 1.7B's from 0.337 to 0.925 and Phi-4-mini's from 0.354 to 1.740, so the three models that leave the OCR almost unchanged under plain decoding become clearly harmful. The clean-text control shows the same effect without any recognition error to repair: guarded decoding raised the CER of error-free input from 0.032 to 0.360 for Gemma-2B-it, from 0.035 to 0.940 for Qwen3 1.7B and from 0.060 to 1.406 for Phi-4-mini. The increase is therefore caused by the decoding constraints, not by the difficulty of the input.

Which constraint does the damage depends on the model. For Gemma-2B-it almost all of it comes from the n-gram ban ($+0.176$ on Tesseract text, against $+0.004$ for the penalty). For Phi-4-mini the penalty alone is catastrophic ($+1.396$, with 28 of 40 pages reaching the token limit) and the n-gram ban moderate ($+0.155$). For Qwen3 1.7B and Llama 3.2 1B each constraint alone raises CER by 0.35 to 0.49 on OCR text and by 0.59 to 0.69 on clean text. An evaluation that uses either default would thus misjudge a copying model, and testing only one of them would not show the size of the problem for every model. An earlier guarded-versus-plain comparison on the 15-page CPU sample (table note) also included two Bengali-pretrained checkpoints: guarded decoding raised the CER of the decoder-only TituLLMs 1B by more than 1, but not that of BanglaT5, an encoder-decoder model whose decoder does not see the input tokens as its own history.

E. Prompt and chunking variants

Table X reports the four general-purpose models under the three prompt variants and the model-independent chunks. No variant brings any model significantly below the raw OCR.

TABLE VI
FULL EVALUATION MATRIX: 40 PAGES, 2 ENGINES, 9 MODELS

Engine	Model	CER	ΔCER [95% CI]	p_{Holm}	WER	ΔWER [95% CI]	p_{Holm}	wMER	Pages better/worse	Trunc.
Tesseract	(no correction)	0.341	—	—	0.827	—	—	0.652	—	—
Tesseract	SymSpell (dictionary)	0.378	+0.037 [+0.031, +0.042]	<0.001	0.918	+0.091 [+0.073, +0.110]	<0.001	0.737^{+}	1/39	**0%**
Tesseract	Gemma-2B-it	0.353	+0.012 [-0.001, +0.029]	1.00^{ns}	0.828	+0.002 [-0.007, +0.012]	1.00^{ns}	0.661	16/16	**0%**
Tesseract	Qwen3 1.7B	**0.337**	**-0.004** [-0.014, +0.004]	1.00^{ns}	**0.799**	**-0.028** [-0.047, -0.013]	0.002	**0.646**	**22/13**	2%
Tesseract	Phi-4-mini	0.354	+0.013 [-0.003, +0.037]	1.00^{ns}	0.822	-0.005 [-0.026, +0.017]	1.00^{ns}	0.671^{+}	18/18	8%
Tesseract	Llama 3.2 1B	0.380	+0.039 [+0.021, +0.058]	0.002	0.803	-0.023 [-0.055, +0.004]	1.00^{ns}	0.681^{+}	7/29	**0%**
Tesseract	BanglaT5	0.860	+0.519 [+0.442, +0.588]	<0.001	1.162	+0.335 [+0.161, +0.525]	<0.001	0.974^{+}	1/39	12%
Tesseract	TituLLMs 1B	1.084	+0.743 [+0.690, +0.793]	<0.001	1.604	+0.778 [+0.652, +0.915]	<0.001	0.927^{+}	0/40	100%
Tesseract	TituLLMs 1B Instruct	1.642	+1.301 [+1.072, +1.545]	<0.001	2.280	+1.454 [+1.164, +1.773]	<0.001	0.945^{+}	0/40	80%
Tesseract	TituLLMs 3B Instruct	0.958	+0.618 [+0.506, +0.736]	<0.001	1.135	+0.308 [+0.139, +0.479]	0.004	0.982^{+}	1/38	8%
EasyOCR	(no correction)	0.297	—	—	0.757	—	—	0.629	—	—
EasyOCR	SymSpell (dictionary)	0.337	+0.040 [+0.033, +0.046]	<0.001	0.852	+0.095 [+0.072, +0.118]	<0.001	0.712^{+}	1/38	**0%**
EasyOCR	Gemma-2B-it	**0.301**	**+0.003** [-0.006, +0.012]	1.00^{ns}	0.756	-0.001 [-0.010, +0.008]	1.00^{ns}	0.636	11/19	**0%**
EasyOCR	Qwen3 1.7B	0.315	+0.017 [+0.004, +0.033]	0.38^{ns}	0.771	+0.013 [-0.012, +0.053]	1.00^{ns}	**0.628**	**13/18**	5%
EasyOCR	Phi-4-mini	0.315	+0.017 [+0.000, +0.046]	0.47^{ns}	0.762	+0.005 [-0.011, +0.029]	1.00^{ns}	0.647^{+}	**13/25**	2%
EasyOCR	Llama 3.2 1B	0.330	+0.033 [+0.012, +0.058]	0.017	**0.748**	**-0.009** [-0.031, +0.011]	1.00^{ns}	0.642	8/29	2%
EasyOCR	BanglaT5	0.831	+0.534 [+0.471, +0.591]	<0.001	1.217	+0.459 [+0.332, +0.603]	<0.001	0.985^{+}	1/39	25%
EasyOCR	TituLLMs 1B	1.033	+0.735 [+0.671, +0.794]	<0.001	1.447	+0.690 [+0.570, +0.816]	<0.001	0.914^{+}	0/40	100%
EasyOCR	TituLLMs 1B Instruct	1.633	+1.336 [+1.116, +1.581]	<0.001	2.289	+1.532 [+1.237, +1.856]	<0.001	0.949^{+}	0/40	92%
EasyOCR	TituLLMs 3B Instruct	0.886	+0.589 [+0.510, +0.663]	<0.001	1.040	+0.283 [+0.203, +0.364]	<0.001	0.979^{+}	0/40	5%

Δ = corrected − uncorrected (positive = worse). CI: paired bootstrap, 10,000 resamples, seed 403. p_{Holm}: two-sided Wilcoxon signed-rank test, Holm-corrected over the 36 CER and WER tests, the primary family of the study; pages whose error rate did not change are dropped by the test (zero differences). ns: not significant. wMER: word match error rate; $^{+}/^{-}$: significantly higher/lower than the raw OCR (exploratory family of 18 tests). Pages better/worse: pages whose CER fell/rose, of 40. Trunc.: pages whose output hit the token cap. **Shaded** : best value in the column among the corrected methods of that engine (lowest error, most pages better, least truncation); it is not compared with the raw OCR.

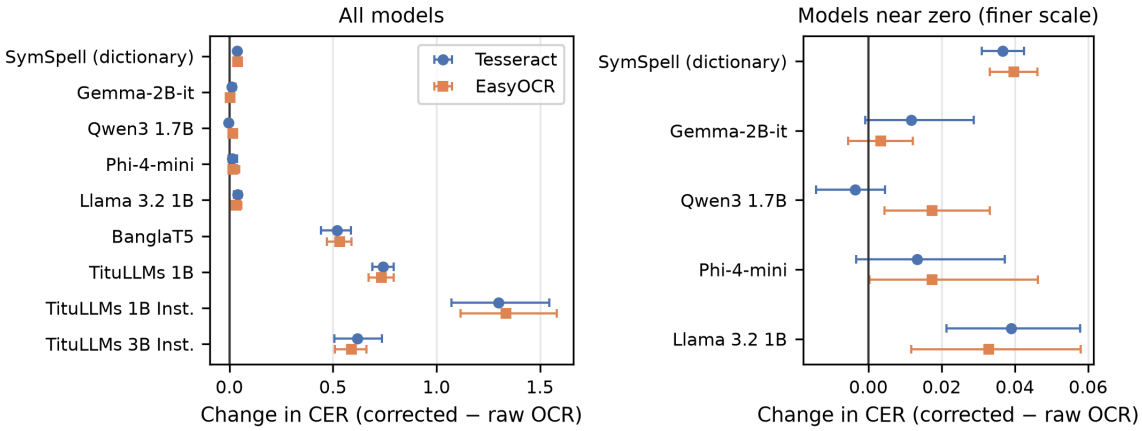


Fig. 3. Mean change in CER (corrected minus raw OCR) per method and engine, with 95% paired-bootstrap intervals over the 40 pages. Values above zero mean the correction made the text worse. Left: all methods. Right: the methods near zero on a finer scale.

The preserve-spelling instruction lowered mean CER in seven of the eight model-and-engine pairs, by up to 0.026 (Llama 3.2 1B on EasyOCR output), but none of these changes is significant. The Bengali prompt and the three worked examples raised CER in most pairs; the increase is significant for Llama 3.2 1B with the Bengali prompt on both engines and with the examples on Tesseract output, and for Phi-4-mini with the Bengali prompt on EasyOCR output. Chunks of at most 300 characters changed mean CER by at most 0.016, with no significant change.

F. The engine-agnostic question

Switching OCR engines changes the size of the effect little: for every method, the mean CER deltas differ by at most 0.035. Gemma-2B-it, Qwen3 1.7B and Phi-4-mini remain nearest the baseline on both engines. The four Bengali-pretrained checkpoints are the most harmful, in the same order. Only Qwen3 1.7B changes direction, from -0.004 on Tesseract to $+0.017$ on EasyOCR; both values are near zero. This is descriptive agreement, not evidence of equivalent effect sizes.

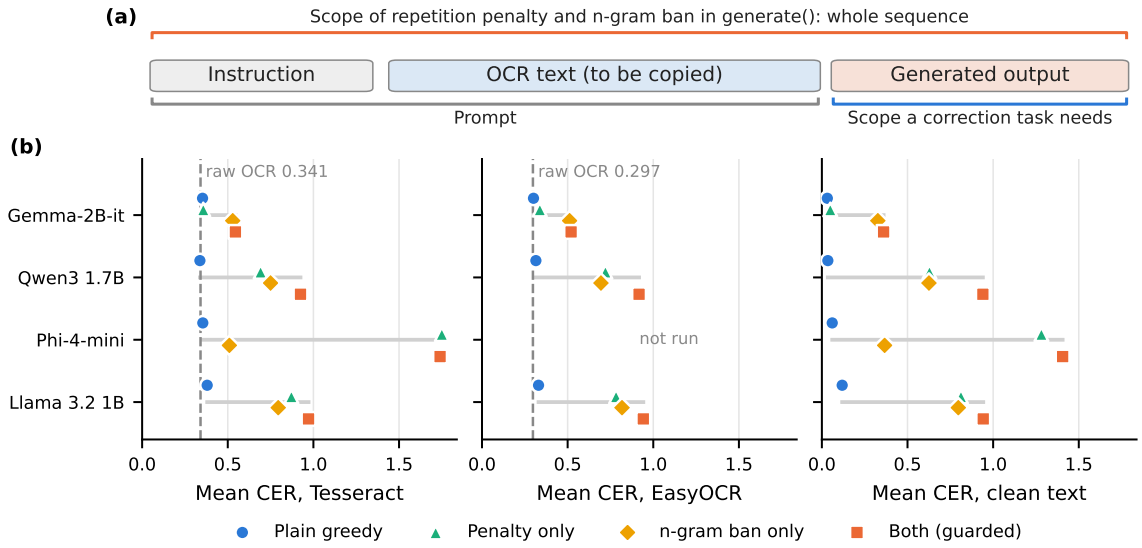


Fig. 4. The decoding pitfall. (a) In the Transformers `generate` function, the repetition penalty and the n-gram ban act on the whole token sequence, which for a decoder-only model includes the OCR text in the prompt, although a correction task needs them, if at all, only on the generated output. (b) Mean CER of the four general-purpose models on the 40 pages under plain greedy decoding, the repetition penalty alone (1.3), the four-gram ban alone, and both (guarded), on Tesseract, EasyOCR and clean-text input; the dashed line is the raw OCR. Markers are offset vertically so that coinciding values stay visible.

TABLE VII
EQUIVALENCE TESTS FOR THE MODELS CLOSEST TO THE RAW OCR

Model	ΔCER	90% CI	TOST p ±0.01	TOST p ±0.02	Better/ worse/same
Tesseract					
Gemma-2B-it	+0.012	[+0.001, +0.026]	0.589	0.150	16/16/8
Qwen3 1.7B	-0.004	[-0.012, +0.003]	0.099	<0.001	22/13/5
Phi-4-mini	+0.013	[-0.001, +0.033]	0.618	0.272	18/18/4
Llama 3.2 1B	+0.039	[+0.024, +0.055]	0.998	0.975	7/29/4
EasyOCR					
Gemma-2B-it	+0.003	[-0.004, +0.011]	0.078	<0.001	11/19/10
Qwen3 1.7B	+0.017	[+0.006, +0.031]	0.830	0.365	13/18/9
Phi-4-mini	+0.017	[+0.001, +0.042]	0.716	0.421	13/25/2
Llama 3.2 1B	+0.033	[+0.015, +0.053]	0.971	0.860	8/29/3

ΔCER: corrected − raw OCR. 90% CI: paired bootstrap. TOST: two one-sided paired t-tests against an equivalence margin of ±0.01 or ±0.02 CER; $p < 0.05$ means the change is within the margin. Not adjusted for multiplicity (exploratory). Last column: pages whose CER fell, rose or stayed the same, of 40.

TABLE VIII
CLEAN-TEXT CONTROL: GROUND TRUTH GIVEN AS INPUT

Model	Mean CER	Pooled CER	WER	wMER	Pages unchanged
SymSpell (dictionary)	0.057	0.057	0.268	0.268	0/40
Gemma-2B-it	0.032	0.028	0.097	0.096	6/40
Qwen3 1.7B	0.035	0.031	0.156	0.154	2/40
Phi-4-mini	0.060	0.097	0.148	0.132	4/40
Llama 3.2 1B	0.119	0.118	0.163	0.157	3/40
BanglaT5	0.833	0.837	1.059	0.937	0/40
TituLLMs 1B	0.988	0.976	1.341	0.810	0/40
TituLLMs 1B Instruct	1.565	1.554	2.129	0.891	0/40
TituLLMs 3B Instruct	0.967	0.954	1.173	0.927	0/40

Each model receives the reference text itself, with the same prompt and plain decoding. Error rates are measured against that reference, so any value above 0 is damage to correct text; WER and wMER are per-page means. Pages unchanged: CER exactly 0 after normalisation.

G. Per-page analysis

Figure 5 plots each page's corrected CER against its baseline CER, and Fig. 6 the change in CER against the baseline CER. Two patterns appear. For the general-purpose models and the dictionary baseline, the points lie along the identity line and corrected CER follows baseline CER almost exactly (Spearman ρ between 0.88 and 1.00 on both engines): these methods leave most of each page as it was, so the page's own difficulty determines its score. Qwen3 1.7B lowered CER on 22 of the 40 Tesseract pages and Gemma-2B-it on 16, but by amounts that their increases on other pages offset. For BanglaT5 and the TituLLMs models, corrected CER is unrelated or only weakly related to baseline CER (ρ between -0.00 and 0.55): their output has a similar error rate whatever the input quality, which suggests that they regenerate the text instead of editing it. Their points form a horizontal band above the identity line in Fig. 5; for BanglaT5, whose corrected CER stays near 0.85 on every page, this band becomes the falling line of Fig. 6.

When the pages are split into thirds by baseline CER, BanglaT5, TituLLMs 1B and TituLLMs 3B Instruct have their smallest increase on the hardest third for both engines; for TituLLMs 3B Instruct on Tesseract output, for example, the increase is $+0.85$ on the easiest third and $+0.39$ on the hardest. The dictionary baseline shows the same pattern on a smaller scale ($+0.047$ against $+0.021$ on Tesseract output). The changes of the general-purpose models stay within 0.07

TABLE IX
DECODING ABLATION ON THE 40 PAGES: REPETITION PENALTY AND n-GRAM BAN, ALONE AND TOGETHER (MEAN CER)

Input	Model	Raw OCR	Plain	Penalty only	n-gram ban only	Both (guarded)	Guarded − plain [95% CI]	Truncated
Tesseract	Gemma-2B-it	0.341	0.353	0.357^{+}	0.529^{+}	0.545^{+}	+0.192 [+0.156, +0.230]	0/0/0
Tesseract	Qwen3 1.7B	0.341	0.337	0.692^{+}	0.750^{+}	0.925^{+}	+0.587 [+0.532, +0.639]	8/3/11
Tesseract	Phi-4-mini	0.341	0.354	1.750^{+}	0.509^{+}	1.740^{+}	+1.385 [+1.175, +1.606]	28/1/24
Tesseract	Llama 3.2 1B	0.341	0.380	0.871^{+}	0.795^{+}	0.972^{+}	+0.592 [+0.508, +0.686]	27/12/27
EasyOCR	Gemma-2B-it	0.297	0.301	0.338^{+}	0.512^{+}	0.520^{+}	+0.220 [+0.187, +0.257]	0/0/0
EasyOCR	Qwen3 1.7B	0.297	0.315	0.721^{+}	0.695^{+}	0.917^{+}	+0.602 [+0.548, +0.653]	14/3/15
EasyOCR	Llama 3.2 1B	0.297	0.330	0.783^{+}	0.818^{+}	0.942^{+}	+0.612 [+0.529, +0.700]	21/21/23
Clean text	Gemma-2B-it	–	0.032	0.049^{+}	0.327^{+}	0.360^{+}	+0.328 [+0.298, +0.359]	0/0/0
Clean text	Qwen3 1.7B	–	0.035	0.628^{+}	0.625^{+}	0.940^{+}	+0.905 [+0.850, +0.966]	10/7/22
Clean text	Phi-4-mini	–	0.060	1.282^{+}	0.366^{+}	1.406^{+}	+1.345 [+1.093, +1.618]	19/2/18
Clean text	Llama 3.2 1B	–	0.119	0.812^{+}	0.797^{+}	0.942^{+}	+0.824 [+0.766, +0.885]	27/17/25

Plain: greedy decoding with only the length cap (the main runs). Penalty: `repetition_penalty`=1.3; n-gram ban: `no_repeat_ngram_size`=4; in a decoder-only model both also act on the prompt, so they penalise copying the OCR text. Clean text: the reference given as input, so any change is damage. $^{+}/^{-}$: higher/lower than plain decoding, Wilcoxon signed-rank test, Holm-corrected over the 33 preset-versus-plain tests (exploratory family). CI: paired bootstrap, 10,000 resamples, seed 403. Truncated: pages that reached the token limit (penalty/n-gram/both), of 40. Phi-4-mini was not run on EasyOCR text (time limit of the GPU session). An earlier ablation of guarded against plain decoding on 15 pages (August OCR, CPU, bfloat16) also covered two Bengali-pretrained checkpoints; guarded minus plain CER on Tesseract and EasyOCR text: TituLLMs 1B +1.07 and +1.56; BanglaT5 -0.01 and -0.02.

TABLE X
PROMPT AND CHUNKING VARIANTS (MEAN CER, 40 PAGES)

Model	Variant	Tesseract	EasyOCR
	Raw OCR	0.341	0.297
Gemma-2B-it	original	0.353	0.301
	preserve spelling	0.350	0.305
	Bengali prompt	0.359	0.326
	three-shot	0.435	0.390
	300-char chunks	0.343	0.300
Qwen3 1.7B	original	0.337	0.315
	preserve spelling	0.335	0.310
	Bengali prompt	0.390	0.308
	three-shot	0.368	0.303
	300-char chunks	0.343	0.299
Phi-4-mini	original	0.354	0.315
	preserve spelling	0.351	0.312
	Bengali prompt	0.402	0.359*
	three-shot	0.447	0.324
	300-char chunks	0.343	0.319
Llama 3.2 1B	original	0.380	0.330
	preserve spelling	0.362	0.304
	Bengali prompt	0.516*	0.459*
	three-shot	0.480*	0.395
	300-char chunks	0.380	0.321

Original: English prompt, chunks of about 250 tokens. Each variant changes one thing: the preserve-spelling instruction, the prompt in Bengali, three worked examples, or chunks of at most 300 characters. *: differs from the original setting, two-sided Wilcoxon signed-rank test, Holm-corrected over the 32 variant comparisons (exploratory family).

of zero in every third. The HIPE-OCRepair task also reports over-correction of low-noise input [20].

H. Robustness of the statistics and the reference

The conclusions do not depend on how the error is measured, how pages are resampled, or how the reference is ordered (Table XI). The pooled CER is lower than the per-page mean (0.298 against 0.341 for raw Tesseract output, 0.286 against 0.297 for EasyOCR), because short pages have higher error rates, but it ranks the methods in the same way, and on it no method is below the raw OCR (Qwen3 1.7B on Tesseract output: 0.299 against 0.298). Grapheme-cluster CER is higher than code-point CER (0.431 and 0.387 for the raw OCR), because one wrong sign makes a whole conjunct wrong; on this metric only Qwen3 1.7B on Tesseract output improves, as reported above. On the bounded cMER (Table XIV) the baselines are 0.290 and 0.260, and 13 of the 18 tests are significant after Holm correction, all increases; they include Llama 3.2 1B on both engines and Phi-4-mini on EasyOCR output (0.279, $p = 0.010$), whose CER changes were smaller. Resampling whole books instead of pages, and grouping pages more coarsely by collection, gives practically the same intervals, as expected with 37 books for 40 pages.

Against a reference rebuilt in the declared reading order, which changes 17 of the 40 references, the raw OCR scores slightly better (CER 0.335 and 0.294 against 0.341 and 0.297), and the same 12 of the 18 CER comparisons are significant, all increases; Qwen3 1.7B on Tesseract output stays at -0.004 ($p = 0.82$). Reference-word recall, which ignores word order and extra text, is 0.431 and 0.484 for the raw OCR. No method raises it significantly: Qwen3 1.7B comes closest ($+0.003$ and $+0.002$), and every other method lowers it significantly, Gemma-2B-it included (-0.017 and -0.019). The absence of a significant improvement therefore persists under the declared reading order and under order-free word recall; a region-matched CER remains untested.

I. CPU and GPU runs

On the five pages whose EasyOCR text was byte-identical between the CPU and the GPU runs, Gemma-2B-it produced the same output on all five, and Qwen3 1.7B and Llama 3.2 1B outputs differed from their CPU counterparts by a CER of at most 0.004. The outputs of TituLLMs 1B and BanglaT5 differed more (CER 0.20 and 0.44 between runs), but their scores agreed (0.945 against 0.954 and 0.806 against

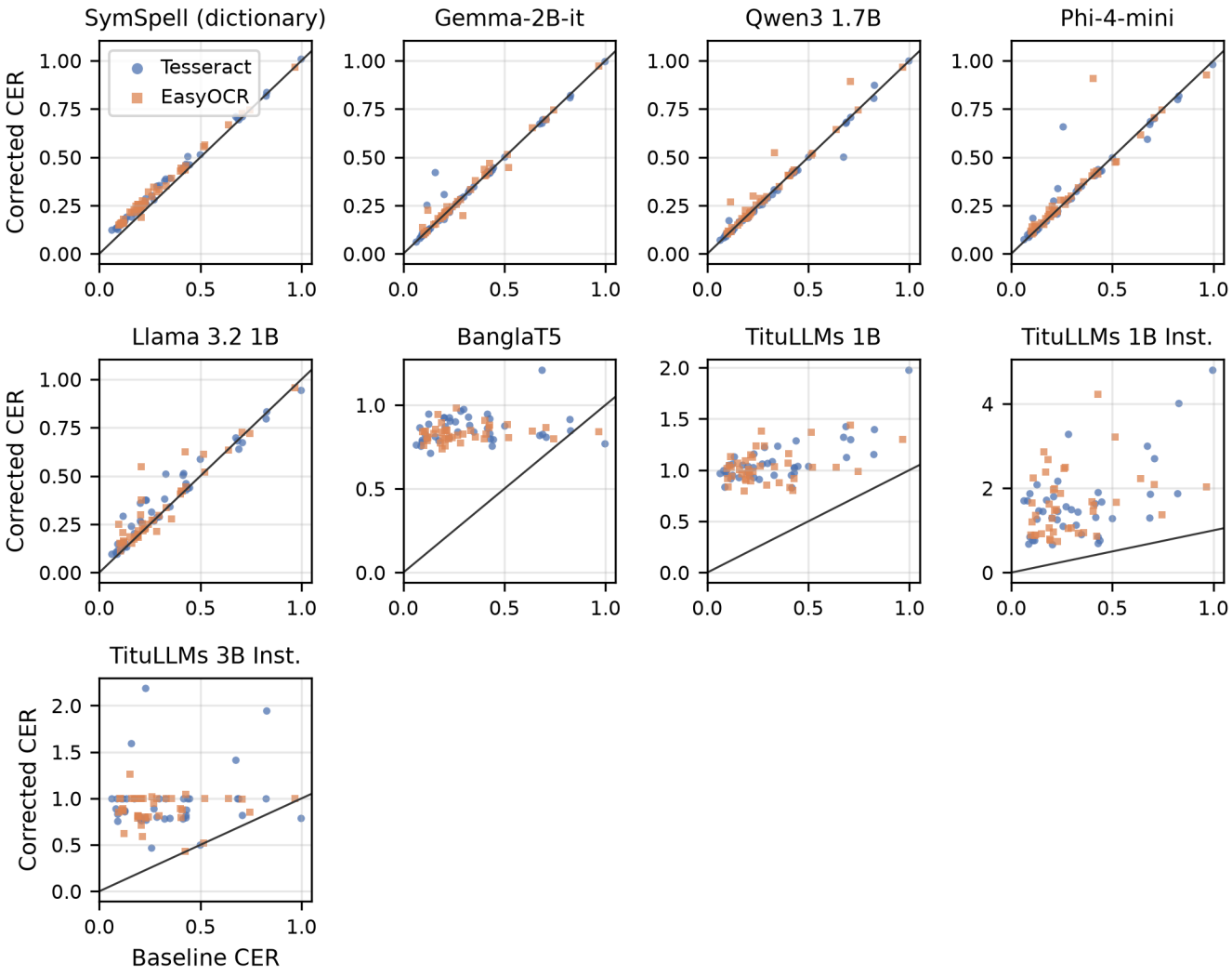


Fig. 5. Corrected CER per page against the page's baseline CER, one panel per method, for Tesseract (circles) and EasyOCR (squares). The diagonal is the identity line: points on it have the same CER as the raw OCR, points below it a lower CER. Vertical scales differ between panels.

0.813). The change of hardware alters individual degenerate outputs, not the conclusions. This check is narrow: hardware and precision change together (bfloat16 on the CPU, float32 on the GPU), only five EasyOCR pages qualify, no Tesseract page does because its text changed between the two OCR runs, and the two float16 models were never run on the CPU.

J. Precision and prompt format

Two choices in the main runs could have made the Bengali-pretrained checkpoints look worse than they are: the numerical precision of the two largest models, and the prompt format (Table XII).

Precision. TituLLMs 3B Instruct, run in float16, returned an empty page in 43 of its 120 outputs (13 on Tesseract text, 14 on EasyOCR text and 16 in the clean-text control). Each empty page took 0.2 to 0.5 s for all its chunks, against a median of 14.5 s for its non-empty pages, so the model ended generation at or near its first token on every chunk. Float16 overflow could cause this, so we reran the model in float32 on two GPUs and recorded its first-token logits in both precisions. The first generated token was the same in both precisions on all 208 chunks, and it was the end-of-sequence token on the same 87. No logit or hidden state was infinite or NaN, and the largest activation was about 1,000, far below the float16 limit of 65,504. The float32 run returned the same empty pages and changed mean CER by at most 0.002 on OCR text (not significant). Phi-4-mini in float32 gave the same CER on Tesseract text (0.354). The empty pages are therefore the model's own behaviour, not a precision artefact.

Prompt format. The base TituLLMs 1B checkpoint received our instruction as plain text with no cue for where its answer should begin. With a completion-style prompt that ends with such a cue, its CER fell from 1.084 to 0.877 on Tesseract text, from 1.033 to 0.861 on EasyOCR text and from 0.988

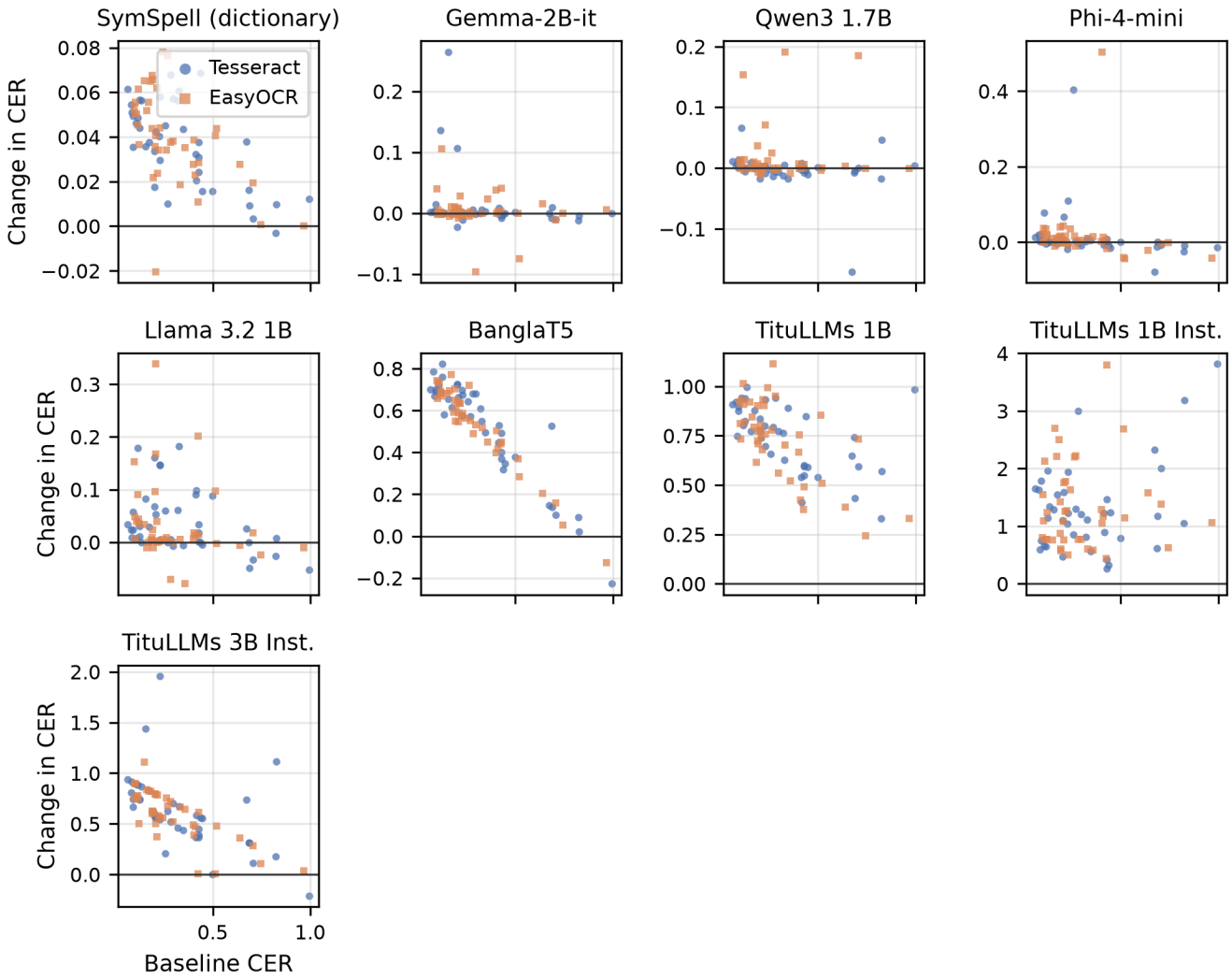


Fig. 6. Change in CER per page against the page's baseline CER, one panel per method, for Tesseract (circles) and EasyOCR (squares). Points above the horizontal line are pages the correction made worse. Vertical scales differ between panels.

to 0.605 on the clean text (all $p < 0.001$), but it remains far worse than the raw OCR ($+0.54$ and $+0.56$ CER on OCR text). Passing BanglaT5's input through the normaliser its model card asks for changed its CER by at most 0.002. The prompt format thus matters for the base checkpoint, but neither check changes a conclusion: every rerun of a Bengali-pretrained checkpoint stays significantly worse than the raw OCR.

V. Failure Analysis

Word retention, edit operations, character classes and truncation distinguish the kinds of damage in the outputs. We also check whether page-level safeguards or confidence gating reduce that damage.

A. Word retention and word-level changes

Table XIII reports how many ground-truth words each output retains. Raw Tesseract output keeps 46% of the distinct ground-truth words and raw EasyOCR output 53%. Averaged over the outputs of both engines, the general-purpose models keep 46% to 49%, the dictionary baseline 37%, and the Bengali-pretrained checkpoints between 3% and 16%. The script shares in the same table are pooled over all output characters, so they describe the text a method produced, not how often it produced any: the output of TituLLMs 3B Instruct is 97% Bengali, but 27 of its 80 outputs are empty. The word-level counts in Table XIV separate correct words that were changed from wrong words that were fixed. Gemma-2B-it broke 4% of the words the raw OCR had right and fixed 0.4% and 0.5% of those it had wrong. Qwen3 1.7B broke 3% and 5% and fixed 3.3% and 3.5%, the closest balance of any method. Phi-4-mini and Llama 3.2 1B broke 8% to 11%, the dictionary baseline 29% and 31%, and the Bengali-pretrained checkpoints between 74% and 99%. The change ratio is below 1 for the general-purpose models and the dictionary (0.1 to 0.4) and

TABLE XI
ROBUSTNESS OF THE MAIN COMPARISON TO THE METRIC, THE RESAMPLING UNIT AND THE REFERENCE

		Grapheme CER		Books	Reading order		Word recall	
Method	Pooled CER	Δ [95% CI]	p_{Holm}	ΔCER 95% CI	ΔCER	p_{Holm}	Δ	p_{Holm}
Tesseract: raw OCR pooled CER 0.298, grapheme CER 0.431, word recall 0.431 [0.377, 0.488]								
SymSpell (dictionary)	0.335	+0.053 [+0.044, +0.061]	<0.001	[+0.031, +0.043]	+0.037	<0.001	-0.102	<0.001
Gemma-2B-it	0.315	+0.010 [-0.002, +0.026]	1.00^{ns}	[-0.000, +0.027]	+0.012	1.00^{ns}	-0.017	0.017
Qwen3 1.7B	0.299	-0.015 [-0.032, -0.004]	0.021	[-0.014, +0.004]	-0.004	0.82^{ns}	+0.003	0.38^{ns}
Phi-4-mini	0.324	+0.012 [-0.013, +0.052]	1.00^{ns}	[-0.003, +0.038]	+0.013	1.00^{ns}	-0.028	<0.001
Llama 3.2 1B	0.343	+0.028 [+0.009, +0.049]	0.08^{ns}	[+0.021, +0.058]	+0.038	0.002	-0.034	<0.001
BanglaT5	0.857	+0.462 [+0.350, +0.565]	<0.001	[+0.440, +0.591]	+0.524	<0.001	-0.386	<0.001
TituLLMs 1B	1.045	+0.807 [+0.734, +0.884]	<0.001	[+0.689, +0.795]	+0.748	<0.001	-0.260	<0.001
TituLLMs 1B Instruct	1.587	+1.453 [+1.186, +1.750]	<0.001	[+1.074, +1.545]	+1.306	<0.001	-0.300	<0.001
TituLLMs 3B Instruct	0.932	+0.558 [+0.431, +0.702]	<0.001	[+0.497, +0.763]	+0.623	<0.001	-0.397	<0.001
EasyOCR: raw OCR pooled CER 0.286, grapheme CER 0.387, word recall 0.484 [0.444, 0.525]								
SymSpell (dictionary)	0.325	+0.054 [+0.044, +0.064]	<0.001	[+0.033, +0.047]	+0.040	<0.001	-0.113	<0.001
Gemma-2B-it	0.299	+0.005 [-0.003, +0.013]	0.84^{ns}	[-0.006, +0.012]	+0.004	0.36^{ns}	-0.019	0.002
Qwen3 1.7B	0.306	+0.011 [-0.003, +0.030]	1.00^{ns}	[+0.005, +0.033]	+0.017	0.20^{ns}	+0.002	0.38^{ns}
Phi-4-mini	0.311	+0.014 [-0.008, +0.050]	1.00^{ns}	[+0.000, +0.046]	+0.018	0.20^{ns}	-0.026	<0.001
Llama 3.2 1B	0.322	+0.021 [+0.001, +0.044]	0.84^{ns}	[+0.012, +0.055]	+0.031	0.012	-0.027	<0.001
BanglaT5	0.832	+0.468 [+0.393, +0.533]	<0.001	[+0.469, +0.593]	+0.537	<0.001	-0.454	<0.001
TituLLMs 1B	1.026	+0.727 [+0.642, +0.809]	<0.001	[+0.666, +0.800]	+0.737	<0.001	-0.291	<0.001
TituLLMs 1B Instruct	1.489	+1.519 [+1.228, +1.866]	<0.001	[+1.113, +1.575]	+1.338	<0.001	-0.355	<0.001
TituLLMs 3B Instruct	0.869	+0.527 [+0.449, +0.603]	<0.001	[+0.511, +0.665]	+0.592	<0.001	-0.449	<0.001

Pooled CER: total character errors over total reference characters. Grapheme CER: CER over extended grapheme clusters; Δ = corrected − raw OCR. Books: 95% interval of the mean change in CER when whole books (37) are resampled instead of pages. Reading order: change in CER against a reference whose regions follow the reading order declared in the PAGE-XML file (17 of 40 references change). Word recall: share of reference word tokens found in the output, ignoring order and extra text; positive Δ = better; the raw value is followed by its 95% bootstrap interval. Each p_{Holm} column is its own exploratory family of 18 tests.

TABLE XII
PRECISION AND PROMPT-FORMAT CHECKS (MEAN CER, 40 PAGES)

		Tesseract		EasyOCR		Clean text	
Check	Model	Main / check	Δ [95% CI]	Main / check	Δ [95% CI]	Main / check	Δ [95% CI]
Float32	TituLLMs 3B Instruct	0.958 / 0.959	+0.000 [-0.001, +0.001]	0.886 / 0.888	+0.002 [-0.000, +0.005]	0.967 / 0.927	-0.040 [-0.132, +0.032]
Float32	Phi-4-mini	0.354 / 0.354	-0.000 [-0.001, +0.000]	–	–	–	–
Completion prompt	TituLLMs 1B	1.084 / 0.877	-0.206* [-0.276, -0.118]	1.033 / 0.861	-0.172* [-0.262, -0.055]	0.988 / 0.605	-0.383* [-0.488, -0.295]
Normalised input	BanglaT5	0.860 / 0.858	-0.001 [-0.014, +0.013]	0.831 / 0.830	-0.001 [-0.004, +0.002]	0.833 / 0.831	-0.002 [-0.007, +0.004]

Float32: instead of float16. Main: the setting of the main runs (float16 for the two models above 3B parameters; each model's shipped chat template; BanglaT5 on unnormalised text). Check: the same model rerun with one change. Δ: check − main; CI: paired bootstrap, 10,000 resamples; *: Wilcoxon signed-rank $p < 0.05$, unadjusted (exploratory). –: not run (Phi-4-mini in float32 on EasyOCR text) or incomplete (on clean text, 1 of 40 pages within the session's time limit). First-token check of TituLLMs 3B Instruct on all 208 chunks: the same first token in both precisions on 208, end-of-sequence first on 87 in both, no non-finite logits or hidden states, largest activation 1000 (float16 limit 65,504). Empty pages (float16/float32, Tesseract, EasyOCR, clean text): 13/13, 14/14, 16/16. Every check of a Bengali-pretrained checkpoint stays significantly worse than the raw OCR (+0.52 to +0.62 CER on OCR text, $p < 0.001$).

between 3.9 and 7.0 for the Bengali-pretrained checkpoints.

B. Edit profiles by model

Table XVI counts substitutions, deletions and insertions per page against the reference; Table XV shows the outputs for two reference lines. In the first example, Gemma-2B-it and Llama 3.2 1B reproduce the Tesseract text unchanged. Qwen3 1.7B repairs a broken word, while the dictionary modernises a historical form. The Bengali-pretrained checkpoints return unrelated text or nothing. In the second example, Qwen3 1.7B repairs one word but damages another.

General-purpose models. Gemma-2B-it, Qwen3 1.7B and Phi-4-mini leave substitutions, deletions and insertions close to the raw OCR's counts (on Tesseract text, Gemma-2B-it: 93, 76 and 64 against 92, 60 and 69). Llama 3.2 1B mainly deletes: deletions rise from 60 to 110 per page on Tesseract text and from 58 to 93 on EasyOCR text.

Dictionary baseline. SymSpell adds substitutions (92 to 111 and 84 to 108), because it replaces historical spellings that are missing from its modern lexicon with modern words.

BanglaT5. Its output is on average 0.45 times the reference length. Deletions rise to 503 and 434 per page and insertions fall to near zero. More than a quarter (28%) of its non-space output characters are neither Bengali nor Latin letters, that is, punctuation, ASCII digits and other symbols. BanglaT5 is not instruction-tuned and was not fine-tuned for correction, which may account for this, but we did not test it.

TituLLMs. The three checkpoints fail in different ways. The base 1B model substitutes and inserts (370 substitutions and 384 insertions per page on Tesseract text), reached the

TABLE XIII
OUTPUT DIAGNOSTICS BY MODEL

Source	Len.	Bn	Latin	Other	GT ret.	Empty
Raw Tesseract	1.03×	93%	0%	7%	46%	0/40
Raw EasyOCR	1.02×	92%	0%	8%	53%	0/40
SymSpell (dictionary)	**1.00×**	93%	**0%**	7%	37%	0/80
Gemma-2B-it	1.00×	92%	**0%**	8%	47%	0/80
Qwen3 1.7B	1.01×	93%	**0%**	7%	**49%**	0/80
Phi-4-mini	1.00×	92%	1%	7%	46%	0/80
Llama 3.2 1B	0.94×	94%	**0%**	6%	46%	0/80
BanglaT5	0.45×	72%	0%	28%	3%	0/80
TituLLMs 1B	1.50×	**97%**	**0%**	**3%**	16%	0/80
TituLLMs 1B Instruct	2.09×	88%	5%	7%	13%	0/80
TituLLMs 3B Instruct	0.50×	97%	0%	3%	4%	27/80

Both engines together (80 outputs per model; 40 per raw OCR row). Len.: output length / reference length, mean over outputs (the raw OCR values are those of Table II). Bn / Latin / Other: share of all non-space output characters, pooled over outputs, so empty outputs do not enter. GT ret.: share of distinct ground-truth words present in the output, mean over outputs (an empty output retains none). Empty: outputs with no text.

generation limit on every page, and repeats itself: in 51 of its 80 outputs a segment of at least 20 characters, split at danda, line breaks and vertical bars, recurs three or more times. The 1B Instruct checkpoint produces output 2.09 times the reference length, with 797 and 722 insertions per page; 33% and 30% of its output consists of runs of at least six characters absent from the input, it reached the generation limit on 80% and 92% of the pages, and it repeats itself in 34 of 80 outputs. The 3B Instruct checkpoint returned an empty page in 27 of its 80 outputs, and in 37 it wrote a Bengali description of the text (for example, that the text appears to be a copy of a historical document) instead of a corrected version; its deletions rise to 432 and 458 per page.

Character classes. The raw OCR is least accurate on Bengali digits (65% and 53% of reference digits substituted or deleted), on the danda (59% for Tesseract; EasyOCR never outputs a danda) and on punctuation, and most accurate on consonants (19% and 14%). Characters inside conjuncts are wrong in 29% and 20% of cases. The general-purpose models raise the error rate of every class slightly; the largest change is Gemma-2B-it on digits (65% to 75% and 53% to 73%). Most of that change is a script conversion: Gemma-2B-it rewrites Bengali digits as ASCII digits, raising the number of ASCII digits in its output from 88 to 336 on Tesseract text and from 420 to 640 on EasyOCR text, while the references contain 585 Bengali digits and 6 ASCII ones. Because digits are a small share of the text, mapping ASCII digits to Bengali digits before scoring changes its CER by only 0.002 and 0.006. Removing the 27 zero-width joiners and non-joiners from the references and outputs changes any method's CER by at most 0.003, and writing khanda-ta as one code point changes nothing, since no text on these pages spells it with a joiner.

Lexicon coverage. 52% of the Bengali word tokens in the references are missing from the 44,335-word lexicon of the dictionary baseline. On pages with more such words, SymSpell breaks a larger share of the words the OCR had right (Spearman $\rho = 0.38$ on both engines, $p < 0.02$), but its CER on the clean text is unrelated to that share ($\rho = 0.08$).

Page types. Lists (contents, indexes, tables) are the hardest pages for both engines, with a raw CER of 0.576 and 0.511, and prose the easiest, at 0.199 and 0.184 (Table XVII). No method improves any page type consistently: the mean changes of the general-purpose models lie between -0.020 and $+0.066$ in every type, the largest decrease being Qwen3 1.7B on Tesseract lists. Three of the four Bengali-pretrained checkpoints, and the dictionary baseline, have their smallest increase on lists, the type with the most errors to begin with, which matches the pattern by difficulty in Section IV. The groups hold 8 to 13 pages, so these figures are descriptive.

C. What the models attend to

The output analyses above show copying and regeneration from the text alone. Token-level Grad-CAM maps (Section III-J) test the same contrast inside the models, on one chunk from each of 10 pages (Table XVIII). The general-purpose models copy 93% to 99% of their output tokens from the OCR text, and for 58% to 78% of the copied tokens the element-wise map at mid depth peaks on the source token itself, against a chance rate of 1% to 2%; at three-quarter depth the rate is 62% to 92%. Their maps put about half of their mass on the OCR text and most of the rest on their own earlier output. The Bengali-pretrained checkpoints copy only 11% to 23% of their output tokens. TituLLMs 1B Instruct relies more on its own earlier output (0.55 and 0.53 of the map) than on the OCR text (0.28 and 0.31), which fits its long, repetitive and invented output described above. The few tokens the TituLLMs checkpoints do copy also tend to peak on their source (41% to 63% at mid depth; BanglaT5 25% and 37%), so the difference lies mainly in how much they copy, not in how they copy. The maps look the same with Tesseract text and with clean text as input.

Literal Grad-CAM does not localise: its map peaks on the copied source token for only 0% to 8% of copied tokens, close to chance. Averaging the gradient over all positions, as Grad-CAM does for the channels of a CNN, keeps one weight per hidden dimension and discards which position the score depends on; the same weights then highlight every token whose hidden state resembles the source. This is the failure that the element-wise variant was proposed to avoid [51]. Phi-4-mini and TituLLMs 3B Instruct were not analysed (Table XVIII).

D. Truncation

TituLLMs 1B reached the generation limit on every page and TituLLMs 1B Instruct on 80% and 92%. The other methods reached it on at most 25% of pages (BanglaT5 on EasyOCR text), and Gemma-2B-it never did. BanglaT5 and TituLLMs 3B Instruct have high error rates with little truncation, so a high error rate does not require truncation.

E. Effect of the safeguards

Table XIX gives both page-level safeguards with their tests. The length rule replaces outputs only for the Bengali-

TABLE XIV
WHAT CORRECTION CHANGED: BOUNDED ERROR, WORD-LEVEL DAMAGE AND FIXES, AND CHANGE RATIO

Engine	Model	cMER	Correct words broken	Wrong words fixed	Change ratio	Invented runs
Tesseract	(no correction)	0.290	—	—	—	—
Tesseract	SymSpell (dictionary)	0.325^{+}	29%	2.9%	0.3	**0%**
Tesseract	Gemma-2B-it	0.303	4%	0.4%	**0.2**	**0%**
Tesseract	Qwen3 1.7B	**0.292**	**3%**	3.3%	0.2	0%
Tesseract	Phi-4-mini	0.305	10%	1.3%	0.2	1%
Tesseract	Llama 3.2 1B	0.338^{+}	11%	1.3%	0.4	0%
Tesseract	BanglaT5	0.847^{+}	96%	1.8%	4.1	0%
Tesseract	TituLLMs 1B	0.705^{+}	74%	**4.6%**	4.9	14%
Tesseract	TituLLMs 1B Instruct	0.744^{+}	79%	1.5%	6.9	33%
Tesseract	TituLLMs 3B Instruct	0.845^{+}	96%	0.3%	4.5	9%
EasyOCR	(no correction)	0.260	—	—	—	—
EasyOCR	SymSpell (dictionary)	0.300^{+}	31%	**6.9%**	0.4	**0%**
EasyOCR	Gemma-2B-it	**0.264**	**4%**	0.5%	**0.1**	**0%**
EasyOCR	Qwen3 1.7B	0.275	5%	3.5%	0.2	0%
EasyOCR	Phi-4-mini	0.279^{+}	11%	1.6%	0.2	0%
EasyOCR	Llama 3.2 1B	0.298^{+}	8%	1.5%	0.4	0%
EasyOCR	BanglaT5	0.828^{+}	99%	2.3%	3.9	**0%**
EasyOCR	TituLLMs 1B	0.677^{+}	76%	4.8%	4.6	13%
EasyOCR	TituLLMs 1B Instruct	0.749^{+}	78%	2.2%	7.0	30%
EasyOCR	TituLLMs 3B Instruct	0.845^{+}	95%	0.3%	4.1	2%

cMER $= (S{+}D{+}I)/(H{+}S{+}D{+}I)$, bounded in $[0, 1]$. Correct words broken: share of ground-truth words the raw OCR had right that the output gets wrong; wrong words fixed: share the raw OCR had wrong that the output gets right (word alignment against the ground truth). Change ratio: CER of the output against the raw OCR, divided by the raw OCR's CER. Invented runs: share of output characters in runs of $\geq$6 characters with no counterpart in the raw OCR. $^{+}/^{-}$: cMER significantly higher/lower than the raw OCR, Wilcoxon signed-rank test, Holm-corrected over the 18 cMER tests (exploratory family). **Shaded**: best corrected model per engine.

pretrained checkpoints: BanglaT5 on 21 and 8 of the 40 pages, TituLLMs 3B Instruct on 18 and 23, and TituLLMs 1B Instruct on 5 and 6. This lowers their CER, for example from 0.958 to 0.660 for TituLLMs 3B Instruct on Tesseract output, but every one of them remains significantly worse than the raw OCR (for that example $+0.319$, 95% CI $[+0.193, +0.460]$, $p = 0.001$ after Holm correction over 18 tests). TituLLMs 1B never triggers the rule, because its outputs stay within 0.35 to 2.8 times the input length, and the general-purpose models never trigger it either, so their results are those of the main comparison. The edit-distance cap brings no method significantly below the raw OCR at any τ in the grid; the lowest is Qwen3 1.7B on Tesseract output at 0.337 to 0.339 against 0.341, not significant. Seven capped results are significantly worse: the dictionary baseline at $\tau \geq 0.1$ on both engines, and Llama 3.2 1B at $\tau = 0.3$ on Tesseract output. For the Bengali-pretrained checkpoints the cap rejects all but at most two pages, so their capped output is essentially the raw OCR.

Confidence gating (Table XX) works at the word level instead of the page level. The engines are unsure of much of the text: 41% of Tesseract words have a confidence below 50 and 77% below 90, and 52% of EasyOCR words lie in boxes below 0.5. Gating removes most of the damage of the harmful methods, for example lowering the CER of TituLLMs 1B on Tesseract output from 1.084 to 0.454 and that of the dictionary baseline from 0.378 to 0.348 at the lowest threshold, but nearly all of them remain significantly worse than the raw OCR. For the general-purpose models gating keeps the CER at or just below the raw OCR on Tesseract output. Qwen3 1.7B at threshold 70 gives the only significant reduction of CER in this study, from 0.341 to 0.337 (-0.004, 95% CI $[-0.006, -0.002]$, $p = 0.038$ after Holm correction over 54 tests), a relative change of about 1%. Gating left 4 of the 40 pages identical to the raw OCR; of the 36 non-zero differences the test uses, 25 are improvements and 11 deteriorations. The gain is negligible and not robust: after Holm correction it is not significant at thresholds 50 or 90 (unadjusted $p = 0.014$ and 0.069), not on EasyOCR output, and the ungated Qwen3 change it starts from is itself not significant. On EasyOCR output, where confidence is per box and therefore coarse, no gated method improves on the raw OCR. Lower thresholds, which allow fewer changes, give the lowest error throughout.

VI. DISCUSSION

For practical use, correction must offer something over retaining raw OCR. Here we consider why it changes correctly recognised text, and why a common decoding default makes that damage look worse.

A. Answers to the research questions

RQ1. No prompted method significantly reduces per-page CER or bounded cMER on either engine, and none beats raw OCR on pooled CER (Tables VI and XI). Gemma-2B-it, Qwen3 1.7B and Phi-4-mini remain close to the baseline, but equivalence within ± 0.02 CER holds only for two of their six cells (Table VII). Qwen3 1.7B gives the only significant gains, all on Tesseract output and all small: WER falls by 0.028 and grapheme-cluster CER by 0.015, and with confidence gating at one threshold CER falls by 0.004 (Table XX).

TABLE XV
TWO REFERENCE LINES WITH THE RAW OCR AND EACH METHOD'S OUTPUT ON TESSERACT INPUT

	Example 1		Example 2	
Reference	শাখায় ঝুলিয়া থাকে, রাত্রি কালে আহার নিমিত্ত বহির্গত		যুদ্ধেস্থিত কৌরবেরে দেখিয়া অর্জুন। অস্ত্রপাত	
Gloss	*"[they] hang from branches, and at night go out for food"*		*"seeing the Kauravas arrayed for battle, Arjuna …"*	
Tesseract (raw)	শাখার ঝুলিয়! থাকে, রাত্রি কালে আহার নিমিত বহির্ত	0.11	বুদ্ধেস্থিত কৌরবেরে দেখিয়। অর্জ্ন। অত্ত্রপাত	0.16
EasyOCR (raw)	শখায় ঝালয় থাকে, বাত্রি কালে অহার নিমিত্ত হয়,	0.23	যুদ্ধেস্থিত কৌরবেরে দেখিয়ী ভজ ন | অস্ত্রপাত	0.16
SymSpell	শাখার ঝুলিয়ে! থাকে, রাত্রি কালে আহার নিহিত বহির্গত	0.13	বুদ্ধেস্থিত গৌরবের দেখিয়ে। অর্জন। অত্ত্রপাত	0.22
Gemma-2B-it	শাখার ঝুলিয়! থাকে, রাত্রি কালে আহার নিমিত বহির্ত	0.11	বুদ্ধেস্থিত কৌরবেরে দেখিয়। অর্জ্ন। অত্ত্রপাত	0.16
Qwen3 1.7B	শাখার ঝুলিয়া থাকে, রাত্রি কালে আহার নিমিত বহির্ত	0.09	বুদ্ধে স্থিত কৌরবেরে দেখিয়া অর্জ্ঞা অত্রপাত	0.20
Phi-4-mini	শাখার ঝুলিয়! থাকে, রাত্রি কালে আহার নিমিত বহির্ত	0.11	বুদ্ধেস্থিত কৌরবেরে দেখিয়। অর্জ্ন। অত্ত্রপাত	0.16
Llama 3.2 1B	শাখার ঝুলিয়! থাকে, রাত্রি কালে আহার নিমিত বহির্ত	0.11	বুদ্ধেস্থিত কৌরবেরে দেখিয়। অর্জ্ন। অত্ত্রপাত	0.16
BanglaT5	প্রাণী।।। এই জাতিভুক্তি.ের অন্তর্ভুক্ত।।	0.81	'অন্তস্থল'। ১৯ '। 'অন্ত'। ১৯ '। 'অন্ত'। ১৯	0.78
TituLLMs 1B	মাংসানী বলিয়া কয়েক জাতি আছে। তাহারা মাংস ভক্ষণ	0.62	তুর্থ বর্ষে পঞ্চম সংখ্যা	0.71
TituLLMs 1B Inst.	পাঠ্য তৈরি করতে পারে না। টেক্সটটি	0.70	যে কোনো বিষয়ে বই, নিবন্ধ বা অন্যান্য সম্পদ	0.78
TituLLMs 3B Inst.	*(empty)*	1.00	একটি চিত্র। এটি শুধুমাত্র	0.78

Pages VT_961_k_0172, 279_2_D_32_0007. Each row shows the span of the output aligned to the reference line, with words at either end dropped while that lowers its CER; words that differ from the reference are in red, and the number at the right is the span's CER against the reference. Glosses are approximate. A dotted circle marks a sign with no letter to attach to (for example a doubled virama) in the text itself.

TABLE XVI
MEAN EDIT OPERATIONS PER PAGE AGAINST THE REFERENCE

Engine	Model	Subst.	Del.	Ins.
Tesseract	(no correction)	92	60	69
Tesseract	SymSpell (dictionary)	111	71	66
Tesseract	Gemma-2B-it	93	76	64
Tesseract	Qwen3 1.7B	92	66	64
Tesseract	Phi-4-mini	102	70	68
Tesseract	Llama 3.2 1B	90	110	53
Tesseract	BanglaT5	126	503	5
Tesseract	TituLLMs 1B	370	19	384
Tesseract	TituLLMs 1B Instruct	344	33	797
Tesseract	TituLLMs 3B Instruct	170	432	88
EasyOCR	(no correction)	84	58	70
EasyOCR	SymSpell (dictionary)	108	68	64
EasyOCR	Gemma-2B-it	83	71	67
EasyOCR	Qwen3 1.7B	92	62	72
EasyOCR	Phi-4-mini	98	68	64
EasyOCR	Llama 3.2 1B	87	93	59
EasyOCR	BanglaT5	181	434	1
EasyOCR	TituLLMs 1B	352	22	385
EasyOCR	TituLLMs 1B Instruct	350	30	722
EasyOCR	TituLLMs 3B Instruct	135	458	50

RQ2. The methods have similar directions and rankings on the two engines, with CER deltas differing by at most 0.035 (Table VI). This supports consistency across the tested engines, but does not establish equal effect sizes.

RQ3. Few recognition errors are repaired (Table XIV). The general-purpose models preserve much of the input, so corrected CER tracks baseline CER page by page (Fig. 5); they recover a few percent of wrong words but break a similar or larger share of correct ones. Their attribution maps confirm the copying from inside the models: most copied tokens rely most on their own source token (Table XVIII). The Bengali-pretrained checkpoints instead regenerate text: they copy few tokens, most correct words are lost, and output error rates vary little with input quality. Damage also occurs when the input is already error-free (Table VIII).

RQ4. Only decoding changed the conclusion (Table IX, Fig. 4): the repetition penalty, the n-gram ban, or both made every general-purpose model significantly harmful, on OCR text and on error-free text alike. A completion prompt lowered the CER of the base TituLLMs 1B checkpoint, and float32 changed nothing, but neither brought a Bengali-pretrained checkpoint near the raw OCR (Table XII). Prompt variants and chunking did not bring any model below the raw OCR (Table X), nor did the page-level safeguards (Table XIX); confidence gating removed most of the damage but gave no meaningful gain (Table XX).

B. Relation to prior work

Our outcome agrees with reported zero-shot failures on Finnish [8] and Hindi speech-recognition output [9]. Noise sensitivity appears even at 70 billion parameters once input CER reaches 0.15 [7], and HIPE-OCRepair reports over-correction of accurate text [20]. Successful studies differ in important ways: they fine-tune for correction [5], [20], use a frontier model on moderately noisy text [21], or supply clean English input [7]. We combine models of 0.25 to 3.8 billion parameters with no task-specific training, a low-resource non-Latin script, historical spelling and input CER of 0.30 to 0.34. The latter exceeds the 0.25 or less reported in the other studies reviewed here. The supervised Bulgarian result shows that correction can help a non-Latin script [6], but its language- and engine-specific training differs from our text-only zero-shot setup. The decoding ablation also shows why a negative result needs care: generation settings can make a model that mostly copies appear destructive. Plain decoding should therefore be checked before attributing such failures to correction itself.

C. Possible explanations

A correction model has to retain most of the input while changing its errors. Our checks help distinguish some reasons

TABLE XVII
MEAN CHANGE IN CER BY PAGE TYPE

	Tesseract				EasyOCR			
Method	title (10)	verse (8)	prose (13)	list (9)	title (10)	verse (8)	prose (13)	list (9)
Raw OCR (CER)	0.319	0.334	0.199	0.576	0.255	0.293	0.184	0.511
SymSpell (dictionary)	+0.037	+0.041	+0.042	+0.025	+0.049	+0.054	+0.036	+0.022
Gemma-2B-it	-0.004	+0.013	+0.012	+0.028	-0.005	-0.001	+0.011	+0.006
Qwen3 1.7B	-0.008	+0.003	+0.007	-0.020	+0.009	+0.027	+0.027	+0.004
Phi-4-mini	-0.002	-0.004	+0.050	-0.007	+0.006	+0.066	+0.015	-0.009
Llama 3.2 1B	+0.045	+0.030	+0.060	+0.010	+0.036	+0.018	+0.048	+0.021
BanglaT5	+0.567	+0.546	+0.650	+0.252	+0.576	+0.544	+0.629	+0.340
TituLLMs 1B	+0.797	+0.723	+0.774	+0.654	+0.740	+0.839	+0.814	+0.524
TituLLMs 1B Instruct	+1.043	+1.597	+1.111	+1.597	+1.677	+1.179	+1.024	+1.546
TituLLMs 3B Instruct	+0.705	+0.675	+0.659	+0.409	+0.587	+0.582	+0.723	+0.403

First row: mean CER of the raw OCR; other rows: mean change in CER (corrected − raw OCR, positive = worse). Page counts in parentheses. Title: title pages; verse: pages whose main text is metrical, including verse with a commentary; prose: running prose; list: contents, indexes, tables and dictionary entries. Descriptive only: the groups are too small for separate tests.

TABLE XVIII
WHAT THE MODELS ATTEND TO: TOKEN-LEVEL GRAD-CAM MAPS

		Share of the map (element-wise, mid)			Map peaks on the copied source token (%)			
Model	Copied tokens (%)	OCR text	Rest of prompt	Own earlier output	Element-wise mid	Element-wise late	Literal mid	Chance
Tesseract text								
Gemma-2B-it	99	0.47	0.09	0.44	76	92	3	1.2
Qwen3 1.7B	96	0.52	0.15	0.33	73	70	2	1.3
Llama 3.2 1B	99	0.46	0.20	0.34	58	68	1	1.3
BanglaT5	11	1.00	–	–	25	18	2	1.5
TituLLMs 1B	16	0.46	0.12	0.43	47	43	2	1.3
TituLLMs 1B Instruct	21	0.28	0.16	0.55	51	78	4	1.5
Clean text								
Gemma-2B-it	96	0.47	0.09	0.44	78	89	4	1.2
Qwen3 1.7B	96	0.51	0.15	0.35	78	67	1	1.3
Llama 3.2 1B	93	0.46	0.20	0.34	65	62	1	1.4
BanglaT5	14	1.00	–	–	37	43	0	1.6
TituLLMs 1B	23	0.44	0.12	0.44	41	36	5	1.4
TituLLMs 1B Instruct	19	0.31	0.16	0.53	63	56	8	1.4

10 chunks per input (one from each of 10 pages, seed 403), 240 generated tokens scored per model and input; plain greedy decoding, the main prompt. Score: the logit of each generated token. Literal: Grad-CAM, gradients averaged over positions; element-wise: gradient × hidden state per position (as in HiResCAM). Mid/late: hidden states at 1/2 and 3/4 of the depth. The first token, special tokens and the predicting position are left out of every map. Copied tokens: output tokens aligned with an identical OCR token. Peak: the largest map value over the OCR text lies within one position of the copied token's source; chance: the rate for a random peak. BanglaT5 is an encoder-decoder model, so its map covers only the encoder input, which is the OCR text. Not run: Phi-4-mini and TituLLMs 3B Instruct (time limit of the GPU session).

for failure, but leave others unresolved.

- *Input noise.* Noise cannot be the whole explanation: with reference text as input, every method still caused damage, and the Bengali-pretrained checkpoints reached CERs of 0.83 to 1.57. Three of the four Bengali-pretrained checkpoints also have their smallest CER increase on the hardest pages and their largest on the easiest. We cannot quantify noise's additional contribution from these checks; that would require a controlled noise sweep.
- *Tokenisation.* Bengali content needs several times the token count of English under English-centric tokenizers [29], [30]. In our evaluation, tokens per character vary fourfold, from 0.29 to 1.22, and tokens per word from 1.9 to 8.0 (Table III). Neither measure gives a simple ranking of outcomes. The TituLLMs Instruct checkpoints have low fertility but high error rates; Llama 3.2 1B and TituLLMs 1B share a tokenizer and the highest fertility, yet one remains near the baseline while the other triples its error rate. Training data and tuning differ too, so tokenizer effects may interact with those differences.
- *Historical spelling.* A model or lexicon built for modern Bengali may mistake older spellings for errors. Our modern dictionary replaces about 31 correct words per page of clean text. Asking models to preserve spelling slightly reduces damage in seven of the eight general-purpose model-and-engine pairs. Mixing old and modern literary registers also caused difficulty for GPT-4o in grammar correction [31]. These observations make historical spelling a plausible explanation, but we have not tested its effect directly.
- *Prompt format and instruction-following.* The instruction-tuned TituLLMs checkpoints often describe or explain

TABLE XIX
POST-HOC SAFEGUARDS: LENGTH RULE AND EDIT-DISTANCE CAP (MEAN CER, 40 PAGES)

	Length rule			Edit-distance cap τ			
Method	CER (fallbacks)	ΔCER [95% CI]	p_{Holm}	0.05	0.1	0.2	0.3
Tesseract: raw OCR CER 0.341							
SymSpell (dictionary)	0.378 (0/40)	+0.037 [+0.031, +0.043]	<0.001	0.344	0.376^{+}	0.378^{+}	0.378^{+}
Gemma-2B-it	0.353 (0/40)	+0.012 [-0.001, +0.029]	1.00^{ns}	0.340	0.340	0.346	0.346
Qwen3 1.7B	0.337 (0/40)	-0.004 [-0.014, +0.005]	0.70^{ns}	0.339	0.339	0.337	0.337
Phi-4-mini	0.354 (0/40)	+0.013 [-0.003, +0.038]	1.00^{ns}	0.340	0.344	0.344	0.344
Llama 3.2 1B	0.380 (0/40)	+0.039 [+0.021, +0.058]	0.001	0.343	0.348	0.355	0.375^{+}
BanglaT5	0.574 (21/40)	+0.233 [+0.151, +0.317]	0.001	0.341	0.341	0.341	0.341
TituLLMs 1B	1.084 (0/40)	+0.743 [+0.691, +0.793]	<0.001	0.341	0.341	0.341	0.341
TituLLMs 1B Instruct	1.305 (5/40)	+0.964 [+0.785, +1.150]	<0.001	0.341	0.341	0.341	0.341
TituLLMs 3B Instruct	0.660 (18/40)	+0.319 [+0.193, +0.460]	0.001	0.341	0.341	0.341	0.341
EasyOCR: raw OCR CER 0.297							
SymSpell (dictionary)	0.337 (0/40)	+0.040 [+0.033, +0.046]	<0.001	0.299	0.335^{+}	0.337^{+}	0.337^{+}
Gemma-2B-it	0.301 (0/40)	+0.003 [-0.005, +0.012]	0.48^{ns}	0.301	0.302	0.301	0.301
Qwen3 1.7B	0.315 (0/40)	+0.017 [+0.004, +0.033]	0.18^{ns}	0.299	0.301	0.305	0.315
Phi-4-mini	0.315 (0/40)	+0.017 [+0.000, +0.045]	0.19^{ns}	0.300	0.302	0.302	0.302
Llama 3.2 1B	0.330 (0/40)	+0.033 [+0.012, +0.058]	0.008	0.300	0.305	0.310	0.314
BanglaT5	0.699 (8/40)	+0.401 [+0.317, +0.483]	<0.001	0.297	0.297	0.297	0.297
TituLLMs 1B	1.033 (0/40)	+0.735 [+0.673, +0.794]	<0.001	0.297	0.297	0.297	0.297
TituLLMs 1B Instruct	1.231 (6/40)	+0.933 [+0.757, +1.110]	<0.001	0.297	0.297	0.297	0.297
TituLLMs 3B Instruct	0.509 (23/40)	+0.212 [+0.124, +0.308]	0.002	0.298	0.298	0.298	0.298

Length rule: the raw OCR replaces an output whose length is outside 0.35–2.8 times the input; fallbacks: pages replaced. Δ: safeguarded minus raw OCR (positive = worse); CI: paired bootstrap, 10,000 resamples; p_{Holm}: Wilcoxon signed-rank test, Holm-corrected over the 18 length-rule tests. Edit-distance cap: the output is kept only if its CER against the raw OCR is at most τ; $^{+}/^{-}$: significantly higher/lower than the raw OCR, Holm-corrected over the 72 cap tests. Both families are exploratory.

TABLE XX
CONFIDENCE-GATED CORRECTION (MEAN CER, 40 PAGES)

	Tesseract (word conf.)			EasyOCR (box conf.)		
Method	50	70	90	0.5	0.7	0.9
Raw OCR	0.341	0.341	0.341	0.297	0.297	0.297
SymSpell (dictionary)	0.348^{+}	0.353^{+}	0.365^{+}	0.315^{+}	0.328^{+}	0.334^{+}
Gemma-2B-it	0.340	0.340	0.341	0.298	0.300	0.302
Qwen3 1.7B	0.338	0.337^{-}	0.338	0.297	0.305	0.306
Phi-4-mini	0.338	0.338	0.340	0.297	0.301	0.308
Llama 3.2 1B	0.342	0.348	0.356	0.303	0.318^{+}	0.325^{+}
BanglaT5	0.440^{+}	0.441^{+}	0.480^{+}	0.375^{+}	0.450^{+}	0.569^{+}
TituLLMs 1B	0.454^{+}	0.511^{+}	0.619^{+}	0.444^{+}	0.674^{+}	0.868^{+}
TituLLMs 1B Instruct	0.427^{+}	0.541^{+}	0.813^{+}	0.567^{+}	0.834^{+}	1.061^{+}
TituLLMs 3B Instruct	0.357	0.346	0.365	0.317	0.382	0.519^{+}

A model's edit is kept only where every OCR word it changes has a confidence below the threshold (column header); elsewhere the raw OCR is kept. $^{+}/^{-}$: significantly higher/lower than the raw OCR, Wilcoxon signed-rank test, Holm-corrected over the 54 gated tests (exploratory family); pages left identical to the raw OCR are dropped by the test.

the input, or return nothing, rather than correct it, and the 1B Instruct checkpoint performs worse than its base model. The prompt format explains only part of this. A completion prompt with an explicit cue for the answer lowered the base checkpoint's CER by about 0.2 but left it far above the raw OCR, and normalising BanglaT5's input changed nothing (Section IV-J). The empty outputs of TituLLMs 3B Instruct are not a float16 artefact: the model ends generation at its first token on the same chunks in both precisions. Instruction tuning that did not cover copy-and-edit tasks remains a plausible cause; the Instruct checkpoints, whose model cards give no prompt format, were tested with their shipped template only.

These explanations and results apply to the checkpoints and configuration tested here. They do not establish how other Bengali-pretrained checkpoints, or fine-tuned versions of these models, would perform.

D. Practical implications

On this corpus, correction gave no meaningful overall gain in code-point CER, while retaining the raw OCR saves about two to fourteen minutes of CPU processing per page, as measured on the 15-page sample (Table IV). The best general-purpose models change CER little; the Bengali-pretrained checkpoints and dictionary increase it. Restricting edits to low-confidence words reduces the measured damage of every method that had increased error, but improves on raw OCR by at most about 1%. For copy-heavy tasks using Transformers generation, repetition penalties and n-gram bans should be avoided or restricted to generated tokens: their default scope includes the prompt. Fine-tuned small models merit evaluation given results in other languages [5], [9], but this experiment does not test them.

E. Threats to validity and limitations

- *Internal validity.* The guarded decoding setting was chosen after seeing development output; the decoding ablation covers the four general-purpose models only (Phi-4-mini not on EasyOCR text). The main runs used a GPU in float32, or float16 for Phi-4-mini and TituLLMs 3B Instruct; a float32 rerun gave the same results, but for

Phi-4-mini only on Tesseract text. The motivating setting is a CPU, and on identical inputs the general-purpose models produced identical or nearly identical outputs on both. The instruction-tuned TituLLMs checkpoints received only their shipped prompt format. The few-shot examples come from development pages, which were also used for debugging.

- *No output filter.* The alignment step that strips preambles was not implemented, so methods that add commentary are penalised for it, the TituLLMs Instruct checkpoints most. Gemma-2B-it, Qwen3 1.7B and Phi-4-mini add almost none and still do not improve CER, so commentary does not explain the absence of improvement.
- *Construct validity.* The engines read the whole page while the ground truth covers the annotated regions, and the reference follows the file order of the regions, not the engines' reading order. Out-of-region text and a different order count as errors, and they need not affect raw and corrected text equally. A reference in the declared reading order and an order-free word recall gave the same conclusions, but we did not evaluate region by region. CER weights every code point equally; grapheme-cluster CER, WER and cMER give the same conclusions except for the small gain of Qwen3 1.7B on Tesseract output.
- *Statistical validity.* With 40 pages from 37 books the intervals are moderately wide, and small effects such as Llama 3.2 1B's became significant only with 40 pages. The 36 CER and WER tests are the primary family; Holm correction controls the family-wise error within each family, not across them, so the other tests are exploratory.
- *External validity.* The pages are early printed books, 1713 to 1914, from one release, so the results may not transfer to modern printed Bengali or to other collections. We tested zero-shot use only, models up to 3.8 billion parameters, two open OCR engines, and four prompts. The page types were labelled from the transcriptions by one annotator, without a second annotator, and each type holds only 8 to 13 pages. EasyOCR reports confidence per text box, so gating on its output is coarse.
- *Experiments not performed.* Five extensions were outside the scope of this zero-shot study on free computing resources: (i) a large open or frontier model on the same pages, which would show how much model scale alone changes the outcome; (ii) fine-tuning, of sequence-to-sequence or byte-level models such as BanglaT5, mT5 or ByT5, or of a small model on synthetic pairs of clean and corrupted text, for which we had neither aligned historical Bengali OCR data nor training compute; (iii) a stronger or cleaner OCR input, which would test whether correction helps once the input CER falls towards the 0.15 or less at which other studies report gains; (iv) direct transcription of the page images by a multimodal model; and (v) a synthetic noise sweep that corrupts the reference at CERs from 0.02 to 0.30 to locate where correction stops helping, of which our clean-text control covers only the noise-free end. The fine-tuning pilot on synthetic pairs is the next experiment we plan.
- *Scope of the attribution analysis.* The Grad-CAM maps cover 10 chunks, six of the eight models and two layers. They show which input tokens an output token depends on, not why a model chooses to copy or regenerate, so the explanations in this section remain hypotheses.

VII. Conclusion

Zero-shot correction by small open models did not improve historical Bengali OCR in this study, and a common decoding default makes such models look far worse than they are: on the same 40 pages it damaged even error-free text. The general-purpose models mostly copied their input, and their attribution maps point at the OCR token being copied; the Bengali-pretrained checkpoints we tested regenerated it, also at full precision and with a completion prompt. For practitioners working under similar conditions, the safe choice is to keep the raw OCR, or to let a copying model change only words the OCR engine marks as uncertain; repetition penalties and n-gram bans should not be applied to the prompt. Whether fine-tuning or larger models change this remains to be tested.

References

[1] D. M. Eberhard, G. F. Simons, and C. D. Fennig, Eds., *Ethnologue: Languages of the World*, 28th ed. Dallas, Texas: SIL International, 2025. [Online]. Available: https://www.ethnologue.com

[2] I. M. Zulkarnain, S. B. Islam, M. Z. A. Z. Farabe, M. M. H. Shawon, J. M. Abedin, B. R. Hasan, M. H. Meghla, I. Shihab, M. T. H. Mobassir, M. N. Ansary, A. Sushmit, and F. Sadeque, "bbOCR: An open-source multi-domain OCR pipeline for Bengali documents," arXiv preprint arXiv:2308.10647, 2023. [Online]. Available: https://arxiv.org/abs/2308.10647

[3] A. K. M. S. A. Rabby, H. Ali, M. M. Islam, S. Abujar, and F. Rahman, "Enhancement of Bengali OCR by specialized models and advanced techniques for diverse document types," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, 2024, pp. 1102–1109. [Online]. Available: https://openaccess.thecvf.com/content/WACV2024W/WVLL/papers/Rabby_Enhancement_of_Bengali_OCR_by_Specialized_Models_and_Advanced_Techniques_WACVW_2024_paper.pdf

[4] V. K. Vershinin, I. V. Khodnenko, and S. V. Ivanov, "Normalization of text recognized by optical character recognition using lightweight LLMs," *Russian Digital Libraries Journal*, vol. 28, no. 5, pp. 1036–1056, 2025.

[5] A. Thomas, R. Gaizauskas, and H. Lu, "Leveraging LLMs for post-OCR correction of historical newspapers," in *Proceedings of the Third Workshop on Language Technologies for Historical and Ancient Languages (LT4HALA) @ LREC-COLING-2024*, Torino, Italia, 2024, pp. 116–121. [Online]. Available: https://aclanthology.org/2024.lt4hala-1.14/

[6] A. Beshirov, M. Dobreva, D. Dimitrov, M. Hardalov, I. Koychev, and P. Nakov, "Post-OCR text correction for Bulgarian historical documents," *International Journal on Digital Libraries*, vol. 26, no. 1, p. 4, 2025.

[7] R. Koynov and T. H. A. Doan, "Opportunities and challenges of LLMs as post-OCR correctors," in *Communication Papers of the 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*, ser. Annals of Computer Science and Information Systems, vol. 45, 2025, pp. 111–118.

[8] J. Kanerva, C. Ledins, S. Käpyaho, and F. Ginter, "OCR error post-correction with LLMs in historical documents: No free lunches," in *Proceedings of the Third Workshop on Resources and Representations for Under-Resourced Languages and Domains (RESOURCEFUL-2025)*, Tallinn, Estonia, 2025, pp. 38–47. [Online]. Available: https://aclanthology.org/2025.resourceful-1.8/

[9] R. Kumar, A. Krishna, G. Ramakrishnan, and P. Jyothi, "Post-ASR correction in Hindi: Comparing language models and large language models in low-resource scenarios," in *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2026, pp. 636–645. [Online]. Available: https://aclanthology.org/2026.eacl-short.45/

[10] U. Pal, P. K. Kundu, and B. B. Chaudhuri, "OCR error correction of an inflectional Indian language using morphological parsing," *Journal of Information Science and Engineering*, vol. 16, no. 6, pp. 903–922, 2000.

[11] M. S. Ahmed, T. Gonçalves, and H. Sarwar, "Improving Bangla OCR output through correction algorithms," in *Proceedings of the 10th International Conference on Software, Knowledge, Information Management and Applications (SKIMA)*. Chengdu, China: IEEE, 2016. [Online]. Available: http://hdl.handle.net/10174/20660

[12] S. S. Joy and S. Shatabda, "BnMMLU: Measuring massive multitask language understanding in Bengali," in *Findings of the Association for Computational Linguistics: ACL 2026*. San Diego, California, United States: Association for Computational Linguistics, 2026, pp. 12 211–12 230.

[13] F. B. Safir, A. Q. Ohi, M. F. Mridha, M. M. Monowar, and M. A. Hamid, "End-to-end optical character recognition for Bengali handwritten words," in *2021 National Computing Colleges Conference (NCCC)*. IEEE, 2021, pp. 1–7.

[14] A. Chaudhury, P. S. Mukherjee, S. Das, C. Biswas, and U. Bhattacharya, "A deep OCR for degraded Bangla documents," *ACM Transactions on Asian and Low-Resource Language Information Processing*, vol. 21, no. 5, pp. 1–20, 2022.

[15] T. T. H. Nguyen, A. Jatowt, M. Coustaty, and A. Doucet, "Survey of post-OCR processing approaches," *ACM Computing Surveys*, vol. 54, no. 6, pp. 1–37, 2021.

[16] G. Chiron, A. Doucet, M. Coustaty, and J.-P. Moreux, "ICDAR2017 competition on post-OCR text correction," in *Proceedings of the 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, 2017, pp. 1423–1428.

[17] C. Rigaud, A. Doucet, M. Coustaty, and J.-P. Moreux, "ICDAR 2019 competition on post-OCR text correction," in *Proceedings of the International Conference on Document Analysis and Recognition (ICDAR)*, 2019, pp. 1588–1593.

[18] S. Rijhwani, A. Anastasopoulos, and G. Neubig, "OCR post correction for endangered language texts," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 5931–5942.

[19] V. Danilova and G. Aangenendt, "Post-OCR correction of historical German periodicals using LLMs," in *Proceedings of the Third Workshop on Resources and Representations for Under-Resourced Languages and Domains (RESOURCEFUL-2025)*, Tallinn, Estonia, 2025, pp. 120–129. [Online]. Available: https://aclanthology.org/2025.resourceful-1.26/

[20] M. Ehrmann, E. Boros, J. Opitz, A. Michail, F. Wagner, and S. Clematide, "ICDAR 2026 HIPE-OCRepair competition on LLM-assisted OCR post-correction for historical documents," in *Document Analysis and Recognition – ICDAR 2026*, ser. Lecture Notes in Computer Science. Springer, 2026, pp. 405–422.

[21] C. M. Viana, "Reframing historical text extraction: A cross-pathway validation of OCR, LLM-assisted correction, and direct multimodal transcription," *Information*, vol. 17, no. 8, p. 722, 2026.

[22] M. A. Levchenko, "Evaluating LLMs for historical document OCR: A methodological framework for digital humanities," in *Proceedings of the First Workshop on Natural Language Processing and Language Models for Digital Humanities (LM4DH 2025)*, Varna, Bulgaria, 2025, pp. 75–85. [Online]. Available: https://aclanthology.org/2025.lm4dh-1.7/

[23] T. Do, D. P. Tran, A. Vo, and D. Kim, "Reference-based post-OCR processing with LLM for precise diacritic text in historical document recognition," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 27, 2025, pp. 27 951–27 959.

[24] A. Bhattacharjee, T. Hasan, W. Ahmad, K. S. Mubasshir, M. S. Islam, A. Iqbal, M. S. Rahman, and R. Shahriyar, "BanglaBERT: Language model pretraining and benchmarks for low-resource language understanding evaluation in Bangla," in *Findings of the Association for Computational Linguistics: NAACL 2022*, Seattle, USA, 2022, pp. 1318–1327.

[25] A. Bhattacharjee, T. Hasan, W. U. Ahmad, and R. Shahriyar, "BanglaNLG and BanglaT5: Benchmarks and resources for evaluating low-resource natural language generation in Bangla," in *Findings of the Association for Computational Linguistics: EACL 2023*, 2023, checkpoint: csebuetnlp/banglat5.

[26] S. K. Nahin, R. N. Nandi, S. Sarker, Q. S. Muhtaseem, M. Kowsher, A. C. Shill, M. Ibrahim, M. H. Menon, T. A. Muntasir, and F. Alam, "TituLLMs: A family of Bangla LLMs with comprehensive benchmarking," in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria, 2025, pp. 24 922–24 940. [Online]. Available: https://aclanthology.org/2025.findings-acl.1279/

[27] A. K. Zehady, S. R. Dipta, N. Islam, S. A. Mamun, and S. Karmaker, "BanglaLlama: LLaMA for Bangla language," in *Proceedings of the Second Workshop on Language Models for Low-Resource Languages (LoResLM 2026)*, 2026, pp. 73–89. [Online]. Available: https://aclanthology.org/2026.loreslm-1.7/

[28] T. Mahfuz, S. K. Dey, R. Naswan, H. Adil, K. S. Sayeed, and H. S. Shahgir, "Too late to train, too early to use? a study on necessity and viability of low-resource Bengali LLMs," in *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*, Abu Dhabi, UAE, 2025, pp. 1183–1200. [Online]. Available: https://aclanthology.org/2025.coling-main.79/

[29] A. Petrov, E. L. Malfa, P. H. S. Torr, and A. Bibi, "Language model tokenizers introduce unfairness between languages," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/74bb24dca8334adce292883b4b651eda-Abstract-Conference.html

[30] O. Ahia, S. Kumar, H. Gonen, J. Kasai, D. Mortensen, N. Smith, and Y. Tsvetkov, "Do all languages cost the same? tokenization in the era of commercial language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 9904–9923. [Online]. Available: https://aclanthology.org/2023.emnlp-main.614/

[31] P. Bhattacharyya and A. Bhattacharya, "Leveraging LLMs for Bangla grammar error correction: Error categorization, synthetic data, and model evaluation," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 8220–8239. [Online]. Available: https://aclanthology.org/2025.findings-acl.431/

[32] C. R. Rahman, M. H. Rahman, S. Zakir, M. Rafsan, and M. E. Ali, "BSpell: A CNN-blended BERT based Bangla spell checker," in *Proceedings of the First Workshop on Bangla Language Processing (BLP-2023)*, Singapore, 2023, pp. 7–17.

[33] M. H. Bijoy, N. Hossain, S. Islam, and S. Shatabda, "A transformer-based spelling error correction framework for Bangla and resource scarce Indic languages," *Computer Speech & Language*, vol. 89, p. 101703, 2025.

[34] C. Clausner, A. Antonacopoulos, T. Derrick, and S. Pletschacher, "ICDAR2019 competition on recognition of early Indian printed documents (REID2019)," in *Proceedings of the 15th International Conference on Document Analysis and Recognition (ICDAR)*, Sydney, Australia, 2019.

[35] T. Derrick and British Library, "Ground truth transcriptions for training OCR of historical Bengali printed texts – Recognition of early Indian printed documents competition – updated with improved XML coordinates," Dataset, version 2. British Library Research Repository. 81 images (400 dpi TIFF) with PAGE XML transcriptions, books printed 1713–1914; Public Domain Mark 1.0, 2020. [Online]. Available: https://bl.iro.bl.uk/concern/datasets/bb125e50-4a09-46e8-904b-6e46cf308d91

[36] R. Smith, "An overview of the Tesseract OCR engine," in *Proceedings of the 9th International Conference on Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.

[37] Tesseract OCR contributors, "Tesseract open source OCR engine, version 5," https://github.com/tesseract-ocr/tesseract. Version 5.3.4 used, 2021.

[38] JaidedAI, "EasyOCR: Ready-to-use OCR with 80+ supported languages," Software, version 1.7.2, 2024. [Online]. Available: https://github.com/JaidedAI/EasyOCR

[39] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.

[40] Hugging Face, "Utilities for generation: Logits processors (RepetitionPenaltyLogitsProcessor, NoRepeatNGramLogitsProcessor)," Transformers documentation and source, `src/transformers/generation/logits_process.py`, 2026. [Online]. Available: https://huggingface.co/docs/transformers/internal/generation_utils

[41] Google DeepMind, "Gemma: Open models based on Gemini research and technology," Google DeepMind, Tech. Rep., 2024, model card: https://huggingface.co/google/gemma-2b-it, revision 9698841.

[42] A. Yang *et al.*, "Qwen3 technical report," arXiv:2505.09388. Model card: https://huggingface.co/Qwen/Qwen3-1.7B, revision 70d244c, 2025.
[43] A. Abouelenin *et al.*, "Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs," arXiv:2503.01743. Model card: https://huggingface.co/microsoft/Phi-4-mini-instruct, revision cfbefac, 2025.
[44] Meta AI, "Llama 3.2: Multilingual large language models," Model card: https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct, revision 9213176, 2024.
[45] W. Garbe, "SymSpell: Symmetric delete spelling correction algorithm," https://github.com/wolfgarbe/SymSpell. Python port symspellpy 6.10.0 used, 2025.
[46] H. Dave, "FrequencyWords: Frequency word lists generated from OpenSubtitles2018," https://github.com/hermitdave/FrequencyWords. Bengali list bn_50k, CC BY-SA 4.0, 2018.
[47] N. Vaessen, "JiWER: Similarity measures for automatic speech recognition evaluation," Software, version 4.0.0, 2022. [Online]. Available: https://github.com/jitsi/jiwer
[48] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
[49] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
[50] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.
[51] R. L. Draelos and L. Carin, "Use HiResCAM instead of Grad-CAM for faithful explanations of convolutional neural networks," arXiv:2011.08891, 2020. [Online]. Available: https://arxiv.org/abs/2011.08891
[52] A. Rahman, "bengali-postocr-sllm: Code, data splits and evaluation for zero-shot post-OCR correction of Bengali," GitHub repository, 2026. [Online]. Available: https://github.com/Ariffurrhmn/bengali-postocr-sllm